

A Corrected and Conditioned Metatheory for Mergeable Replicated Data Types

Sal / FP Launchpad (KC + Claude)

July 20, 2026

Abstract

Mergeable replicated data types reconcile divergent states with a ternary merge relative to a common ancestor. We give a corrected, mechanized metatheory for this setting. A reachable LCA invariant repairs a gap in the published induction; a concrete legal execution refutes the unconditioned soundness argument; and canonical history states expose the contextual Join obligation actually needed by adequacy. We develop flat and conditioned contracts, honest-reachability and witness-indexed discharge routes, observational quotients, product composition, virtual-LCA semantics, commit garbage collection, a local asynchronous garbage collector that refines the no-GC protocol, and a stability interface for representation changes. The claimed results are kernel checked in Lean 4, with executable countermodels for invalid stronger statements.

Contents

Scope and correctness interface	2
I The metatheory	3
1 Introduction	3
2 Mergeable replicated data types, by definition and example	5
3 The ternary setting	6
4 Three facts the metatheory must respect	9
4.1 The LCA lemma is a reachability invariant — with a gap in the published proof	10
4.2 A fully LCA-legal execution defeats the soundness proof	10
5 Canonical states and the ternary Join Lemma	12
6 Why the contract is a delta contract	14
7 The arbitration contract	16

8 Auditing the published definition: four strikes and the absorber	19
9 Deriving lo: the eight verification conditions	20
10 Adequacy	26
11 The catalogue, discharged	28
12 The conditioned metatheorem	29
13 The conditioned verification conditions	30
14 The metatheorem	32
15 The factored discharge route: honest reachability and per-configuration joins	33
16 Virtual LCAs: merging without a common-ancestor version	40
17 Commit GC: the keep set and the GC-safety theorem	45
18 The stability VC: verified GC callbacks	51
II Audit and mechanization scope	57
19 Mechanization scope	57

Scope and correctness interface

This paper proves the history-to-canonical-state layer of the Sal correctness stack. Its primary interface is a contextual ternary Join lemma: if two canonical branch states and their canonical common-ancestor state are merged in a reachable configuration, the result is the canonical state of the union. The conditioned and honest-reachability variants permit applicability, witness, and safety hypotheses to depend on the actual configuration. Observational quotients and product kits lift that result without conflating raw representation equality with client-visible equality. Virtual LCAs, commit collection, and stability epochs extend the same interface to runtime representation changes. Datatype-specific sequence semantics are applications of this interface and lie outside this paper.

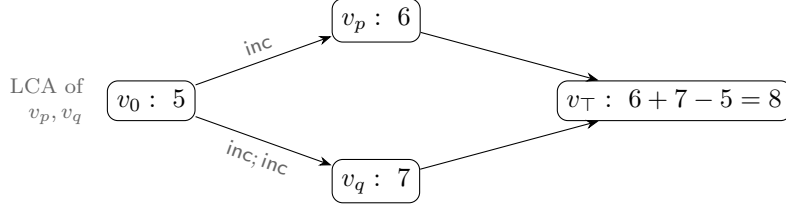


Figure 1: The counter MRDT. Both branches inherit the 5 from the fork point; the LCA argument lets the merge count each branch’s *delta* exactly once. No binary merge of 6 and 7 alone can recover 8: the information “how much is shared” lives in the DAG, and the LCA argument is how the data type reads it.

Part I

The metatheory

1 Introduction

An MRDT execution looks like a git history: replicas fork, apply operations locally, and merge. Unlike a state-based CRDT, whose binary merge must reconcile two states with no memory of their common past, an MRDT’s merge receives a third argument — the state at the *lowest common ancestor* (LCA) of the two versions being merged — and may use it to compute *deltas*: what each side did since the fork. The paradigmatic example is the counter,

$$\text{mergeL}(l, a, b) = a + b - l,$$

which adds the two sides’ increments-since- l instead of double-counting their common past (Figure 1).

The Neem methodology (OOPSLA 2025) promises a clean division of labour: the data-type designer discharges a fixed catalogue of equations over **do**, **mergeL** and the conflict-resolution relation **rc** — mostly by SMT — and a soundness meta-theorem, proved once, converts those VCs into *RA-linearizability* of every reachable configuration. This part documents what a Lean 4 mechanization of that meta-theorem, in the full **ternary**, **version-DAG** setting, found: the meta-theorem’s central induction is unsound as written, and for LCA-sensitive data types several of the catalogue’s merge VCs are false in principle when quantified over all states. It also develops the corrected metatheory: a set-relative linearization order, canonical states $\sigma(E)$ (each event set determines a unique state), and a ternary *Join Lemma* whose induction consumes a small, contextual VC surface. Its contributions:

1. **The paper’s LCA lemma is true but its proof has a gap** (§4.1): $L(v_\ell) = L(v_1) \cap L(v_2)$ holds, but only as a *reachability invariant* of the store, whose proof needs a generator-uniqueness induction the paper silently assumes. We mechanize the transition system and the invariant.
2. **The soundness proof has a fully LCA-legal defeater** (§4.2): a reachable configuration of a legal add-wins MRDT — its final merge sitting at an honest LCA — on which every bottom-up peel of the paper’s derivation rules is stuck. The meta-theorem must be rebuilt, not patched.

3. **The lattice contract does not transfer; a delta contract does** (§6): the lattice laws are equations over *free* tuples — the wrong instances for an LCA-aware merge. Side-idempotence $\text{mergeL}(l, a, a) = a$ with l free fails for the counter $(2a - l)$, but its only DAG-honest instance has $l = a$ (the LCA of a version with itself *is* that version), where it holds trivially; and when two *distinct* histories reach equal side states, $2a - l$ is the *correct* merge — both branches’ increments count — not an idempotence failure. The induced orders degenerate, and the honest associativity law has *four distinct LCAs* and holds only on feasible tuples. The moral: the algebra of an LCA-aware merge is an algebra of *causal histories*, not of concrete states. What replaces the lattice laws is a two-law *delta contract* — redistribution identities in which the LCA slot does, by arithmetic, the job idempotence does order-theoretically for CRDTs.
4. **Eight VCs suffice** (§9–§10): three relational VCs on rc (one of them carrying a different-replica guard — same-replica pairs are ordered by program order, and demanding rc cover them is unsatisfiable for real data types), merge symmetry, three feasibility-conditioned delta laws, and one contextual *causal-delta* equation. The adequacy theorem is per *version*: every version in the store, LCAs included, linearizes.
5. **The VC set is validated on the production catalogue** (§11): the flat production MRDTs shipped in Sal — OR-set, an optimized OR-set, enable-wins flag, grow-only set and map, increment-only and PN counters, a tombstone RGA, Peritext, the multi-valued register and the add-wins priority queue — are discharged end-to-end, kernel-checked; the sequence data types with physical deletion go through the conditioned framework and its factored discharge route (§12–§15, Part II). The sweep produces an empirical *class map* of which data types need which strength of contract, with two findings: tombstones simplify the metatheory, and commutativity class and delta class are independent axes.
6. **The metatheorem factors** (§15): conditioning splits as a generic honest-reachability metatheorem times a per-datatype join discharge (`HonestReach`, `JoinLemma3At`), with a uniform honesty shape (a client-checkable predicate holding of each event at the fold of its causal past). Three join species are observed in production (conditioned commutation, flat commutation, direct witness), and the mergeable queue, whose conflict graph admits no rc assignment at all, is the primary example in the direct-witness species: Peepul’s merge is its own linearization certificate. The same route carries Part II’s embedded-chain RGA.
7. **The model meets its runtime** (§16–§18): the criss-cross gate on Merge is lifted by a recursive virtual-LCA rule whose synthesized base provably carries exactly the intersection event set, with no new per-datatype VC for join-lemma datatypes; commit GC is proved safe against a keep set read off pairwise head intersections, with a countermodel showing head-sync is load-bearing; and state-level compaction (the GC callback into the datatype) gets a six-clause stability VC whose metatheorem makes compaction observationally invisible, with the OR-set as first instance and a refuted naive gate as the shaping countermodel.

Everything stated as a theorem below is mechanized with zero `sorry`s; §19 maps statements to Lean names and files (foundations under `Framework/`, metatheorems under `Metatheory/`, negative results under `Refutations/`, the per-RDT instances under `MRDT_Instances/`, the investigation record under `Development/`; each cited theorem kernel-checked).

2 Mergeable replicated data types, by definition and example

Data type and implementation. A replicated data type of *type* $\tau = \langle O_\tau, Q_\tau, Val_\tau \rangle$ exposes a set of update operations O_τ , a set of query operations Q_τ , and a domain of query return values Val_τ .

Definition 2.1 (flat MRDT signature). *An MRDT implementation of τ is a tuple*

$$\mathcal{D} = \langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{query}, \text{rc} \rangle,$$

where Σ is the space of replica states with initial state $\sigma_0 \in \Sigma$; $\text{do} : \Sigma \times \mathcal{E} \rightarrow \Sigma$ applies an event — an update-operation instance $e = (t, r, o)$ carrying a globally unique timestamp t , its origin replica r , and the operation $o \in O_\tau$; $\text{mergeL} : \Sigma^3 \rightarrow \Sigma$ is the three-way merge, whose first argument is the state at the lowest common ancestor of the two versions being merged; $\text{query} : \Sigma \times Q_\tau \rightarrow Val_\tau$ reads a state; and $\text{rc} : \mathcal{E} \times \mathcal{E} \rightarrow \{\text{Fst_then_snd}, \text{Snd_then_fst}, \text{Either}\}$ is the conflict-resolution relation, consulted to order concurrent non-commuting operations (it is the data-type designer's declaration of who wins a race; *Snd_then_fst* is the conventional mirror of *Fst_then_snd*, and production instances do return it, so the VCs below only ever test the *Fst_then_snd* direction); we write $e_1 \xrightarrow{\text{rc}} e_2$ for $\text{rc}(e_1, e_2) = \text{Fst_then_snd}$.¹ *MRDTSig*

Queries never change state and play no role in the metatheory; the theory below reads the implementation as $\langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc} \rangle$.

Describing an MRDT: the counter. To describe an MRDT one gives the five components and, when operations can race, says who wins. The increment-only counter of Figure 1 is the smallest complete example:

$$\Sigma = \mathbb{Z}, \quad \sigma_0 = 0, \quad \text{do}(\sigma, (t, r, \text{inc})) = \sigma + 1, \quad \text{mergeL}(l, a, b) = a + b - l, \quad \text{rc} = \emptyset.$$

All operations commute, so rc is empty, yet the merge still reads the shared past off its LCA: $-l$ subtracts the common history that $a + b$ alone would double-count. The pattern recurs throughout the catalogue.

A data type with races: the OR-set. The add-wins observed-remove set stores tagged elements: **add** inserts a tag unique to the adding event, **rem** deletes exactly the tags it has *seen*. Sequentially, a remove after an add deletes it; concurrently the designer declares the add the winner: $\text{rem} \xrightarrow{\text{rc}} \text{add}$. The merge keeps a tag iff both sides have it (nobody deleted it) or some side added it since the fork:

$$\text{mergeL}(l, a, b) = (a \cap b) \cup (a \setminus l) \cup (b \setminus l).$$

The LCA here plays the role a tombstone set plays for the CRDT OR-set: deletion is absence *relative to* l , and the state needs no graveyard.

¹In the mechanization the ternary signature *MRDTSig* extends the binary *CRDTSig* (which contributes Σ , σ_0 , do , query , rc , and a *binary merge*) with the field *mergeL* and one reuse contract, *merge_init_slice*: $\forall a b, \text{mergeL}(\sigma_0, a, b) = \text{merge}(a, b)$, pinning the inherited binary merge to the $l := \sigma_0$ slice so the binary corpus runs unchanged on the init-LCA sub-family.

The generalized signature. Both examples’ operations make sense on *every* state: any state can be incremented, any tag can be added. Such data types are *flat*, and for them the metatheory of §3–§11 quantifies its VCs over all states. But a data type whose operations carry *preconditions* — insert under a node that must exist, delete a node that must be live — breaks that quantification: its commutation equations are simply false at nonsense states. The full signature therefore extends the tuple.

Definition 2.2 (conditioned MRDT signature). *A conditioned MRDT implementation extends the flat tuple with two declarations,*

$$\mathcal{D} = \langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc}, \text{Inv}, \text{app} \rangle, \quad \text{Inv} : \Sigma \rightarrow \text{Prop}, \quad \text{app} : \mathcal{E} \times \Sigma \rightarrow \text{Prop},$$

*together with an observational equivalence $\approx \subseteq \Sigma \times \Sigma$: **Inv** is a state invariant (a shape over-approximation of reachability), **app** an applicability predicate (when an event is sensible at a state — it may read the event’s timestamp, so it is a generation-time guard, not a shape predicate), and $\approx$ an observational equivalence (what queries can distinguish; representation residue is quotiented away). In the mechanization **Inv** and **app** are fields of the signature; $\approx$ is not — the quotient layers carry it as a parameter — which is why the tuple above says “together with”. **ConditionedMRDTSig***

Commutation is then required only at **Inv**-states where both events are applicable: the derived *conditioned commutation* (**ConditionedMRDTSig.commutatesOn**) demands $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ only for σ with **Inv**(σ), **app**(e_1, σ), and **app**(e_2, σ). Its VCs are formalized in §13. Flat data types take **Inv** = **app** = $\top$ and $\approx = =$, collapsing the general signature to the plain one — one framework, of which §3–§11 is the flat instance and §12 onwards the general case.

The running hard example: sequences with physical deletion. The datatype class that forces the general signature is the replicated list underlying collaborative text editing. Characters are nodes identified by timestamps, each inserted *after* an anchor node; production RGAs keep deleted nodes as *tombstones* because later concurrent inserts may still anchor on them, and a *tombstone-free* design deletes physically. Its operations are precondition-laden: an insert names an anchor that must exist, a delete names a live node, and the flat update layer quantifies its commutation VCs over every state, including nonsense states no execution produces, where those equations are false. Worse, what makes reachable-state commutation true is a property of the *execution that generated the operation* (the issuer saw the anchor live), not of the state the operation later meets, so it is not expressible as a $\forall \sigma$ equation over the signature alone. This is the reason the general signature exists: **Inv** and **app** name the sensible states and operations, an honesty discipline makes generation-time facts available to the proofs, and $\approx$ absorbs representation residue. Part II presents the embedded-chain RGA, the sequence datatype that holds the repository’s canonical seat, together with its end-to-end discharge through the factored route of §15; a predecessor design that carried recorded ancestor paths and rehomed orphans on delete was proved RA-linearizable in the same framework but refuted at its sequential specification (Part III).

3 The ternary setting

This section fixes the model: the data-type signature, the replicated store and its transitions, and the specification the metatheorem must deliver. The definitions are the paper’s, with one

correction flagged where it occurs.

Signature. An MRDT is $\langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc} \rangle$ as in Definition 2.1, read flat for now ($\text{Inv} = \text{app} = \top$, $\approx = =$; the general case resumes in §12). We write $e(\sigma)$ for $\text{do}(\sigma, e)$, and $e_1 \rightleftharpoons e_2$ when $e_1(e_2(\sigma)) = e_2(e_1(\sigma))$ for every σ ($\text{CRDTSig.commutates}$).

Formal preliminaries. Events are triples $e = (t, r, o)$ with $\text{time}(e) = t$, $\text{rep}(e) = r$, $\text{op}(e) = o$. We write $\text{fold}(\sigma, \pi)$ for the left fold $e^n(\dots e^1(\sigma)\dots)$ of a list $\pi = e^1 \dots e^n$ (applySeq). Two events are *distinct* (distinctOps) when their *timestamps* differ — timestamps are globally unique, so between events of one execution this is inequality of the events — and *cross-replica* (differentReplicas) when their origin replicas differ. A list π *enumerates* a set E (listPermOf) when it is duplicate-free and contains exactly the elements of E ; π *respects* a relation R (respects) when no pair occurs against R : $i < j$ implies $\neg R(e^j, e^i)$ (stated elementwise; R need not be transitive). The *recorded past* of an event (downset), relative to the ambient configuration’s visibility, is

$$\downarrow e := \{ x \mid x = e \vee x (\xrightarrow{\text{vis}} \wedge \not\rightarrow)^+ e \},$$

the transitive closure into e of visible *non-commuting* predecessor edges together with e itself: the fragment of e ’s causal past that a fold can still see (commuting predecessors are invisible to states).

Convention (plumbing). Three pieces of Lean plumbing are folded out of every display below, once and for all: (i) every signature carries decidable equality on Σ and on the operation alphabet (fields dec_state , dec_op of CRDTSig), and framework files use classical choice; (ii) the σ -layer definitions (the set-relative order, canonical states, the join and the merge VCs) are stated over the *replica-keyed core* of a configuration — its events and visibility only — and a ternary configuration C enters them through the forgetful **Configuration.core**; the displays do not distinguish C from its core; (iii) where a display quantifies over configurations, the six core invariants of Definition 3.1 are ambient; (iv) the pruning operators of §17 carry two definedness side conditions (the pinned root and every allocated head lie outside the dropped set), discharged once by the GC run invariant, and displays write the pruned configuration without repeating them. No other hypothesis is elided.

Executions. A configuration carries a *store*: a DAG of versions, each holding a state and the set of events that produced it, with per-replica heads; transitions fork a replica, apply a fresh event at a head, merge two heads through their LCA, or query. The model is fixed by the next two definitions.

Definition 3.1 (ternary configuration). *A configuration C of $\mathcal{D}$ consists of:*

- *a replica-keyed core: partial per-replica maps N (head state) and L (observed event set) and a visibility relation $\xrightarrow{\text{vis}}$ over events, subject to six invariants — N and L share a domain; every $\xrightarrow{\text{vis}}$ -edge has its source, and its target, observed at some replica; causal closure ($a \xrightarrow{\text{vis}} b$ and b observed at r imply a observed at r); timestamp uniqueness across observed events; causal monotonicity ($a \xrightarrow{\text{vis}} b$ implies $\text{time}(a) < \text{time}(b)$, the Lamport-clock guarantee); and same-replica totality (two distinct observed events of one replica are $\xrightarrow{\text{vis}}$ -comparable). $E(C)$ denotes the set of events observed anywhere (**Configuration.events**);*

- a ranked version store: a partial map **ver** from versions ($v \in \mathbb{N}$, the allocation rank) to pairs $(\sigma_v, L(v))$ of a state and an event set, a partial per-replica head-version map, and a parent list **parents**(v), subject to: parents have strictly smaller rank; version 0 is $(\sigma_0, \emptyset)$; a replica's head version carries exactly that replica's $(N(r), L(r))$; every registered state satisfies **Inv**; and the lca-events shape — for all registered v_1, v_2, v_ℓ with v_ℓ an LCA of v_1, v_2 , $L(v_\ell) = L(v_1) \cap L(v_2)$. The last clause is carried as a structure field; §4.1 proves it maintainable, so it is an invariant of the system, not an assumption (Remark 4.4).

Ancestry and LCAs are read off the parent lists: **Reaches**(a, v) is the reflexive-transitive closure of the parent edge $p \in \text{parents}(v)$ (**Reaches**), and v_ℓ is an LCA of v_1, v_2 when it reaches both and dominates every common ancestor (**IsLCA**):

$$\text{Reaches}(v_\ell, v_1) \wedge \text{Reaches}(v_\ell, v_2) \wedge \forall w. \text{Reaches}(w, v_1) \wedge \text{Reaches}(w, v_2) \Rightarrow \text{Reaches}(w, v_\ell).$$

Configuration

Definition 3.2 (the ternary transition system). The labelled transition system **Step3** has four rules:

- **CREATEREPLICA**(r): requires $N(r)$ undefined; installs $N(r) := \sigma_0$, $L(r) := \emptyset$, head at version 0; store and visibility unchanged.
- **APPLY**(t, r, o): requires r 's head version v with **ver**(v) = (σ, E_v) , store-wide timestamp freshness (no event observed anywhere in C , and no event of any registered version, has timestamp t), and a fresh slot $v_{\text{new}} > v$; allocates $v_{\text{new}} := (e(\sigma), E_v \cup \{e\})$ for $e = (t, r, o)$ with parent $[v]$, extends $\xrightarrow{\text{vis}}$ by $E_v \times \{e\}$, and moves r 's head to v_{new} .
- **MERGE**(r_1, r_2) — the rule the metatheory lives in:

$$\frac{\begin{array}{c} \text{head}(r_1) = v_1 \quad \text{head}(r_2) = v_2 \quad \text{ver}(v_i) = (\sigma_i, E_i) \quad \text{IsLCA}(v_1, v_2, v_\ell) \quad \text{ver}(v_\ell) = (\sigma_\ell, E_\ell) \\ \text{ver}(v_m) \text{ unallocated} \quad v_1 < v_m \quad v_2 < v_m \end{array}}{\text{ver}(v_m) := (\text{mergeL}(\sigma_\ell, \sigma_1, \sigma_2), E_1 \cup E_2), \quad \text{parents}(v_m) := [v_1, v_2], \quad \text{head}(r_1) := v_m}$$

with visibility unchanged. Merge is gated on the existence of an LCA; criss-cross configurations disable it (§16 lifts the gate).

- **QUERY**(r, q): reads $N(r)$; configuration unchanged.

The initial configuration (**initConfig**) has one replica at σ_0 observing no events and the one-node DAG $\{0\}$; reachability is along **Step3** (**labeledTS3**). **Step3**

RA-linearizability, per version. Write $e_1 \parallel e_2$ when neither $e_1 \xrightarrow{\text{vis}} e_2$ nor $e_2 \xrightarrow{\text{vis}} e_1$ (concurrency).

Definition 3.3 (linearization order, set-relative). For an event set E , the order lo^E puts e_1 before e_2 iff

$$(e_1 \xrightarrow{\text{vis}} e_2 \wedge e_1 \not\stackrel{\text{rc}}{\rightarrow} e_2) \vee (e_1 \parallel e_2 \wedge e_1 \xrightarrow{\text{rc}} e_2 \wedge \neg \exists e_3 \in E. e_2 \xrightarrow{\text{vis}} e_3 \wedge e_2 \not\stackrel{\text{rc}}{\rightarrow} e_3) :$$

a visible non-commuting pair is ordered by visibility; a concurrent pair whose conflict $\mathbf{rc}$ resolves is ordered by $\mathbf{rc}$ — unless e_2 is already absorbed (overwritten) by a later non-commuting event of E . The paper’s order $\mathbf{lo}$ is the variant whose absorber clause ranges over the whole configuration (*Sal.Emulation.lo*; ternary mirror *Configuration.lo*); since a $\mathbf{lo}$ -edge asserts no absorber anywhere, $\mathbf{lo} \subseteq \mathbf{lo}^E$ pointwise, for every E (*loOn_of_lo*), and $\mathbf{lo}^E$ is antitone in E — growing the set only adds absorbers, hence removes edges (*loOn_mono*). *loOn*

Definition 3.4 (arbitration, and RA-linearizability against it). An arbitration $\mathbf{arb}$ assigns to each event set E a strict order $\mathbf{arb}(E)$ on its operations that orders every visible non-commuting pair ($e_1 \xrightarrow{\text{vis}} e_2 \wedge e_1 \not\Rightarrow e_2 \Rightarrow e_1 \mathbf{arb}(E) e_2$). A list $\pi = e^1 e^2 \dots e^n$ witnesses a version holding state s and event set E against $\mathbf{arb}$ when (i) π enumerates E without repetition; (ii) π extends $\mathbf{arb}(E)$: $i < j$ implies $\neg(e^j \mathbf{arb}(E) e^i)$; and (iii) π folds to the state: $e^n(\dots e^2(e^1(\sigma_0))\dots) = s$, read up to the data type’s observational equivalence $\approx$ in the general signature (flat data types instantiate $\approx =$). A configuration is RA-linearizable against $\mathbf{arb}$ when every version in the store — not only the replica heads; LCAs are historical versions and the merge reads them — admits a witness.

The canonical arbitration is $\mathbf{lo}$ (Definition 3.3), built from the conflict resolver $\mathbf{rc}$. Unqualified, “RA-linearizable” means against $\mathbf{lo}$, and $\mathbf{rc}$ enters the definition through this one instance only: the property itself is not tied to $\mathbf{rc}$. §7 gives the contract an arbitration must meet for a data type to be provably RA-linearizable against it, and exhibits the LWW register linearized against its timestamp order — an arbitration $\mathbf{rc}$ cannot express. *IsRALinearizable3Arb* (against a given $\mathbf{arb}$), *IsRALinearizable3* / *IsRALinearizable3Eq* (at $\mathbf{arb} = \mathbf{lo}$)

Why a set-relative order? One definitional correction is forced at the outset: for every internal use, the absorber clause must range over the event set of *the version being linearized*, not over the whole configuration — an event can gain absorbers from another branch, silently cancelling an order edge inside a version whose own state depends on it (the defeater of §4.2 realizes exactly this). We therefore construct witnesses respecting $\mathbf{lo}^E$ throughout; since $\mathbf{lo} \subseteq \mathbf{lo}^E$, every such witness also extends the paper’s order, and Definition 3.4 is delivered exactly as the paper states it.

Running examples. Three data types exercise every corner of the theory: the **counter** and the **OR-set** of §2, and the **enable-wins flag** — per-replica pairs (counter, flag); enabling bumps your own counter and sets your flag; disabling clears all flags; the merge compares each replica’s counter against the LCA’s to decide whether an enable “happened since the fork” on that replica. That the flag’s merge *compares* counters rather than accumulating sets will be visible in the metatheory (§11).

4 Three facts the metatheory must respect

Before building anything, we record two boundary facts about the paper’s development: its store-level LCA lemma is true but under-proved, and its soundness induction is refuted by a fully legal execution. Together they fix the shape of the corrected metatheory — resting on a mechanized store invariant (§4.1) and built around the join rather than the sides (§4.2).

4.1 The LCA lemma is a reachability invariant — with a gap in the published proof

Everything above silently used the LCA lemma: in every reachable configuration, an LCA’s event set is the intersection of the two sides’. This is what lets merge VCs be stated over *event sets*: the state the merge receives as l is the canonical state of exactly the intersection of the two sides’ histories. The lemma is true — but it is a *global induction over the reachable store*, not a local fact. The invariant the induction threads is:

Definition 4.1 (store invariant). *For a version store $(\text{ver}, \text{parents})$, the store invariant demands:*

- *parents allocated: $p \in \text{parents}(v)$ implies $\text{ver}(p)$ is defined;*
- *event monotonicity: $p \in \text{parents}(v)$, $\text{ver}(p) = (\sigma_p, E_p)$, and $\text{ver}(v) = (\sigma_v, E_v)$ imply $E_p \subseteq E_v$;*
- *origin (generator uniqueness): if $\text{ver}(v) = (\sigma, E)$ and $e \in E$, then there is a version v_0 with $\text{ver}(v_0) = (\sigma_0, E_0)$ and $e \in E_0$ such that every version w with $\text{ver}(w) = (\sigma_w, E_w)$ and $e \in E_w$ satisfies $\text{Reaches}(v_0, w)$.*

StoreInv

Lemma 4.2 (LCA lemma). *Let $(\text{ver}, \text{parents})$ satisfy the store invariant (Definition 4.1), let v_ℓ be an LCA of v_1, v_2 , and let all three versions be registered: $\text{ver}(v_1) = (\sigma_1, E_1)$, $\text{ver}(v_2) = (\sigma_2, E_2)$, $\text{ver}(v_\ell) = (\sigma_\ell, E_\ell)$. Then*

$$E_\ell = E_1 \cap E_2.$$

lca_events_of_storeInv

Theorem 4.3 (the store invariant is a reachability invariant). *For every $\mathcal{D}$ with $\text{Inv}(\sigma_0)$ and every configuration C reachable from the initial configuration in the ternary transition system, $(C.\text{ver}, C.\text{parents})$ satisfies the store invariant.*

storeInv_reachable

The published proof asserts the generator-uniqueness step (the *origin* clause) without the induction that sustains it; Theorem 4.3 is that induction, mechanized (`storeInv_init`, `storeInv_step`). An erratum-sized gap: the statement survives, the proof as written does not. The LCA, when it exists, is unique — two LCAs dominate each other and the rank order is antisymmetric (`isLCA_unique`). (A related boundary fact: *criss-cross* merges can defeat the *existence* of an LCA — they gate merge-enabledness, not the lemma. §16 lifts the gate.)

Remark 4.4 (the lca-events field). *In the mechanization the LCA lemma’s shape is also carried as a field of **Configuration** (the lca-events clause of Definition 3.1), and `merged_store_lca_events` / `applied_store_lca_events` prove the field maintainable under Merge and Apply for stores satisfying the invariant — so Step3’s merge rule is never blocked by it. The “reachability invariant” claim above is the pair: the field plus Theorem 4.3’s maintenance. The field is an invariant of the system, not an assumption smuggled into the model.*

4.2 A fully LCA-legal execution defeats the soundness proof

The generic half of the paper’s contract is a *bottom-up linearization* theorem: given linearization witnesses for the two sides of a merge, construct one for the merged version by repeatedly *peeling* a final event through `mergeL` using the catalogue’s interchange VCs. The version DAG’s

discipline — every merge sitting at an honest LCA — keeps the peel supplied with peelable events only under an additional assumption. A staging replica suffices to realize the intersection of two histories as an honest version, and the merge induction is stuck.

Example 4.5 (the defeater). *The data type is a one-key add-wins skeleton with tagged adds: $\sigma = (A, D) \in 2^{\mathbb{T}} \times 2^{\mathbb{T}}$ with live set $A \setminus D$; add_t inserts its tag into A ; rem kills everything it currently sees ($D := A \cup D$ — the state-dependence is what makes $\text{add} \not\equiv \text{rem}$); concurrently $\text{rem} \xrightarrow{\text{c}} \text{add}$ (add wins); the merge is componentwise union, ignoring its LCA argument. A legal MRDT. Replica p adds (A_p), replica q adds (A_q). A staging replica s merges both one-add versions, creating v_s with $L(v_s) = \{A_p, A_q\}$. Now p removes (R_p , seeing only A_p) and then merges with v_s ; symmetrically q removes (R_q) and merges with v_s . Figure 2 shows the DAG. The final merge of the two heads finds $\text{LCA} = v_s$, and $L(v_s) = \{A_p, A_q\} = L(\text{head}_p) \cap L(\text{head}_q)$ — the configuration is fully legal. Now count last events. p ’s head lives on exactly A_q ’s tag, so a state-correct witness must run R_p before A_q , and neither R_p nor A_p can come last: every linearization of p ’s head ends in A_q . Symmetrically, every linearization of q ’s head ends in A_p . Yet in the merged version every tag is dead, so its linearizations must end in one of the removes. The events the induction must peel are buried in the very sides that own them; no ordering of the paper’s bottom-up rules applies.*

The two removes commute ($\text{rem} \not\equiv \text{add}$ but $\text{rem} \rightleftharpoons \text{rem}$), which invites a particular rescue attempt: the paper’s BOTTOMUP-2-OP rule (`inter_lca_2op`) admits the commuting case and could, one hopes, peel a remove last. It cannot, for a reason that is a one-line state fact. To peel o last from a side, that side’s state must be an o -image — of the form $\text{do}(\sigma', o)$ with o outermost. But in the add-wins skeleton *every* rem -output has empty live set ($\text{rem}(A, D) = (A, A \cup D)$), whereas both heads carry a live tag (head_p lives on A_q , head_q on A_p — the opposite add, merged in *after* the local remove). So neither head is a rem -image, and no bottom-up rule can extract a remove from it. The tempting linearization $[A_p, R_p, A_q, R_q]$ is a valid witness of the *merged* version, but its restriction to head_q folds to the all-dead state, not head_q ’s actual state (live A_p): it is not assembled from a state-correct side witness, which is exactly what the induction requires. All three facts are mechanized (file `Refutations/InterLca2op_Defeater_Arbiter.lean`):

Proposition 4.6 (the defeater’s three mechanized facts). *In the add-wins skeleton of Example 4.5 — writing $\text{live}(A, D) = A \setminus D$, with events $A_p = (1, p, \text{add})$, $R_p = (3, p, \text{rem})$, $A_q = (2, q, \text{add})$, $R_q = (4, q, \text{rem})$, and $\text{head}_p, \text{head}_q$ the two head states of Figure 2:*

1. **(rem-images are all-dead)** for every state σ and every event e with $\text{op}(e) = \text{rem}$:
 $\text{live}(e(\sigma)) = \emptyset$. *awset_rem_output_empty*
2. **(no 2-OP rem-peel)** no instantiation of the BOTTOMUP-2-OP merge shape peels a remove from either head:

$$\neg \exists a, b \in \Sigma, o_l, o_1, o_2 \in \mathcal{E}. \text{op}(o_1) = \text{rem} \wedge o_1(o_l(a)) = \text{head}_p \wedge o_2(o_l(b)) = \text{head}_q,$$

and symmetrically with head_p and head_q interchanged.
no_inter_lca_2op_rem_peel_of_defeater

3. **(a valid merged witness that is not side-correct)** the list $w = [A_p, R_p, A_q, R_q]$ satisfies $\text{live}(\text{fold}(\sigma_0, w)) = \emptyset$ and $\text{fold}(\sigma_0, w) = \sigma_{v_{\top}}$ (the merged state) — a valid witness of the merged version — yet its restriction to head_q ’s event set, $w \upharpoonright = [A_p, A_q, R_q]$, has $\text{fold}(\sigma_0, w \upharpoonright) \neq \sigma_{\text{head}_q}$ and $\text{live}(\text{fold}(\sigma_0, w \upharpoonright)) \neq \text{live}(\sigma_{\text{head}_q})$. *crack1_witness*

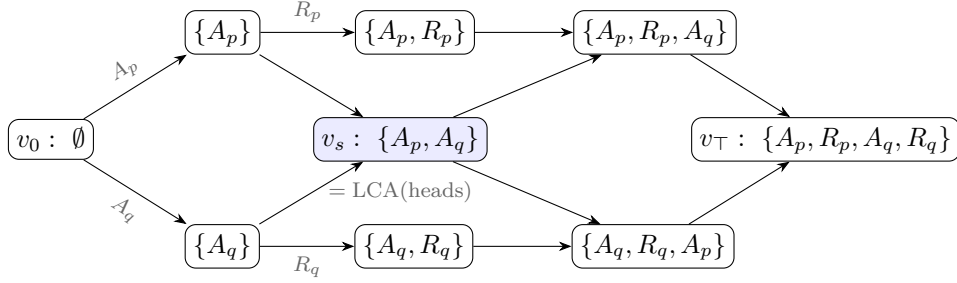


Figure 2: The defeater (Example 4.5). Nodes are versions labelled by their event sets; the shaded version v_s , created by a staging replica, realizes $L(\text{head}_p) \cap L(\text{head}_q)$ as an honest store version, so the final merge is LCA-legal. The removes R_p, R_q — the only events a bottom-up derivation may peel last — are buried mid-history on their own sides.

The consequence for the design: the metatheorem cannot be patched peel-by-peel — it must be rebuilt on canonical states and a Join Lemma, with merge-side VCs that are statements *about the join*, not about how either side’s state factors.

5 Canonical states and the ternary Join Lemma

We now rebuild — new proof architecture, and with it a *new set of verification conditions* to replace the paper’s catalogue. The lesson of the defeater is architectural: witnesses for a merged version cannot be manufactured out of witnesses for its sides, so the corrected metatheory must not traffic in witness lists at all. The rebuild has two layers. The *update layer* makes states, not sequences, the objects of the induction: it assigns to every event set E a single well-defined state, its *canonical state* (Definition 5.2 below). The *merge layer* is then one equation between canonical states — the ternary *Join Lemma* — and establishing it from per-data-type VCs becomes the entire burden of the metatheorem (§6–§9).

The update layer comes first, and a structural observation makes it cheap: this half of the corrected theory never mentions the merge. Its one theorem is stated over the update signature $\langle \Sigma, \sigma_0, \text{do}, \text{rc} \rangle$ alone:

Theorem 5.1 (convergence). *Assume the update-layer VCs (Flat VCs 1–3 of §9; Lean bundle *UpdateVCs*). Let C be a configuration, $E \subseteq E(C)$ an event set, and π_1, π_2 two enumerations of E each respecting lo^E . Then, for every start state σ ,*

$$\text{fold}(\sigma, \pi_1) = \text{fold}(\sigma, \pi_2).$$

The paper-facing statement is the $\sigma := \sigma_0$ instance; the start-state genericity is consumed by the adequacy induction (Lemma 14.5).

convergence_on_u

This is where the set-relative reading of §3 earns its keep: with the paper’s configuration-wide order in place of lo^E , the theorem is *false* — the defeater’s heads are already counterexamples, since two order-respecting replays of the three events of head_p reach different states. Convergence licenses a definition:

Definition 5.2 (canonical state). A state s is a canonical state of the finite event set E in C , written $\text{Can}_C(E, s)$, when some enumeration of E respecting lo^E folds to it from the initial state:

$$\text{Can}_C(E, s) :\iff \exists \rho. \rho \text{ enumerates } E \wedge \rho \text{ respects } \text{lo}^E \wedge \text{fold}(\sigma_0, \rho) = s.$$

Under the update-layer VCs the definite article is earned: a canonical state exists whenever $\xrightarrow{\text{vis}}$ is transitive and irreflexive on C , $E \subseteq E(C)$, and E admits an enumeration (*isCanonicalState_exists_u*; the required acyclicity of lo^E is Flat VC 2’s contribution), and it is unique for $E \subseteq E(C)$ (*isCanonicalState_unique_u*, from Theorem 5.1). We then write $\sigma(E)$ for the canonical state, with $\sigma(\emptyset) = \sigma_0$, and σ_v for the state a version v happens to hold in the store — proving that the two coincide is precisely the metatheorem’s job. Displays below keep hypothesis positions in the predicate form $\text{Can}_C(E, s)$, which does not presuppose uniqueness.

IsCanonicalState

With canonical states in hand, witness lists disappear as promised: a version linearizes iff $\sigma_v = \sigma(L(v))$ — the witness, when one is owed, is any respecting enumeration of $L(v)$. The induction can therefore maintain a state-level invariant (“every version holds σ of its events”) instead of constructing sequences. The one genuinely ternary statement is what the merge must produce:

Definition 5.3 (ternary Join Lemma). Call an event set E weakly closed in C when it is backward-closed under non-commuting visibility: $a \xrightarrow{\text{vis}} b$, $a \not\sqsubseteq b$, and $b \in E$ imply $a \in E$. The ternary Join Lemma at C (*JoinLemma3At*, the framework’s per-configuration hook) is the statement: for all event sets E_1, E_2 and states s_0, s_1, s_2 , if

- (i) $\xrightarrow{\text{vis}}$ is transitive and irreflexive on C ;
- (ii) $E_1, E_2 \subseteq E(C)$;
- (iii) E_1 and E_2 are each weakly closed in C ; and
- (iv) $\text{Can}_C(E_1 \cap E_2, s_0)$, $\text{Can}_C(E_1, s_1)$, $\text{Can}_C(E_2, s_2)$,

then

$$\text{Can}_C(E_1 \cup E_2, \text{mergeL}(s_0, s_1, s_2)).$$

The ternary Join Lemma (*JoinLemma3*) is its closure over all configurations. In σ -notation the conclusion reads $\sigma(E_1 \cup E_2) = \text{mergeL}(\sigma(E_1 \cap E_2), \sigma(E_1), \sigma(E_2))$; but the closure premise (iii) is part of the contract, not scaffolding — the enable-wins-flag asymmetry of §10 turns on exactly that clause.

JoinLemma3, JoinLemma3At

By Lemma 4.2 the first argument is exactly the state the store hands the merge. The adequacy proof (§10) is then an induction over the transition system maintaining “every version holds the canonical state of its event set”; the merge case is the Join Lemma. What remains is the question this part exists to answer: which per-data-type VCs make the Join Lemma true?

6 Why the contract is a delta contract

Where should the laws that prove the Join Lemma come from? The obvious supplier is the CRDT tradition. For state-based CRDTs — binary merge, no LCA — merge-coherence is lattice-flavoured: associativity, idempotence, inflation of updates. None of the three transfers to the ternary merge: their free-tuple forms are the wrong equations for an LCA-aware merge, and their honest instances are too weak to power a lattice-style proof. The shape of what fails to transfer reveals the contract that succeeds.

Idempotence: the free-tuple form is the wrong equation. The *diagonal* law $\text{mergeL}(a, a, a) = a$ — the paper’s own `MERGEIDEMPOTENCE` — holds (for the counter: $a + a - a = a$), but it is not the law a lattice-style proof consumes. The binary architecture uses idempotence to absorb a duplicated *side*, $\text{merge}(a, a) = a$, with nothing tying the two copies to a common past; the ternary analogue would be $\text{mergeL}(l, a, a) = a$ with l *free*. But quantifying l freely asks the merge about tuples the version DAG never produces. Merging a version with *itself* has $l = a$ — the LCA of a and a is a — where the law collapses to the diagonal and holds. And when two *distinct* histories happen to reach equal side states, feasibility puts l strictly below them and the counter’s $\text{mergeL}(l, a, a) = 2a - l$ is the *correct* answer: both branches’ increments count. So there is no idempotence *failure* — there is no meaningful side-idempotence *law*: its honest instances are the trivial diagonal, and demanding it unconditionally would exclude precisely the LCA-sensitive data types the ternary theorem exists for.

The general principle: the algebraic properties of an LCA-aware merge are properties of the *causal history*, not of the concrete state. Two sides merge idempotently when their *histories* coincide — which is what forces $l = a$ — not when their states happen to be equal; equal states with different histories rightly merge differently. The lattice laws go wrong exactly by quantifying over states while forgetting the histories that produced them, and every law that survives below is stated over history-indexed data: canonical states $\sigma(E)$, feasible tuples (the LCA slot carrying the intersection history), downset deltas.

No usable order. The lattice-style workhorse order $x \sqsubseteq y := \text{merge}(x, y) = y$ has ternary candidates $x \sqsubseteq_l y := \text{mergeL}(l, x, y) = y$, but for the counter this relation degenerates to $x = l$; no antisymmetry survives to power order-theoretic reasoning. The single contextual VC below is accordingly an *equation*, not an inequality.

Associativity is the wrong target. The natural guess — $\text{mergeL}(l, \text{mergeL}(l, a, b), c) = \text{mergeL}(l, a, \text{mergeL}(l, b, c))$ — assumes one LCA for all three merges. In an execution, re-associating a three-way reconciliation involves *four* distinct LCAs:

$$\text{mergeL}(l_{(ab)c}, \text{mergeL}(l_{ab}, a, b), c) \stackrel{?}{=} \text{mergeL}(l_{a(bc)}, a, \text{mergeL}(l_{bc}, b, c)),$$

with $l_{ab} = \sigma(E_a \cap E_b)$, $l_{(ab)c} = \sigma((E_a \cup E_b) \cap E_c)$, and so on. For the counter — whose canonical states are additive measures over event sets — the two sides come to $a + b + c$ minus the two LCA measures, and they agree because

$$(E_a \cup E_b) \cap E_c = (E_a \cap E_c) \cup (E_b \cap E_c) \implies \text{both LCA-sums} = |E_{ab}| + |E_{ac}| + |E_{bc}| - |E_{abc}|$$

by inclusion–exclusion. The law holds, but only *through the set-algebra of intersections*: over four unconstrained states it is false. Honest associativity is inherently a feasible-tuple fact — and **the corrected induction never needs it**. Every merge the induction forms already sits at its honest LCA; the only laws consumed are two *redistribution* identities that equate two honestly-formed merges:

Definition 6.1 (the delta contract).

$$\text{LOCAL-REDISTRIBUTE: } \text{mergeL}(l, \text{mergeL}(m, x, c), y) = \text{mergeL}(m, \text{mergeL}(l, x, y), c),$$

$$\begin{aligned} \text{REDISTRIBUTE: } \text{mergeL}(\text{mergeL}(m, x_0, c), \text{mergeL}(m, x_1, c), \text{mergeL}(m, x_2, c)) \\ = \text{mergeL}(m, \text{mergeL}(x_0, x_1, x_2), c). \end{aligned}$$

Both laws are quantified over all states l, m, x, x_i, c, y — this is the unconditional form, the one the group and lattice classes satisfy outright. *DeltaVCs3*

Read c as a *delta relative to m* (in the induction, c will be “apply event e to its causal past”). LOCAL-REDISTRIBUTE says a delta application commutes past an enclosing merge through one branch slot. REDISTRIBUTE says a delta carried by *all three* components extracts *once* — the LCA slot cancels the duplicates by arithmetic. This is the delta contract’s quiet coup: cancelling the duplicated shared contribution is exactly the job idempotence performs in lattice-based convergence arguments, now done without any idempotence — which is why the counter sails through (REDISTRIBUTE for the counter is the identity $\sum x_i + 3c - 3m - (x_0 + c - m) - \dots$ collapsing on both sides).

Where the contract holds unconditionally — and where it cannot. The two laws hold *on all states* exactly for the group-like class (the counter) and the lattice-like, LCA-blind class (grow-only structures). They are *unconditionally false* for the OR-set: take an element tag present in c and l but in neither branch — the two sides of LOCAL-REDISTRIBUTE then disagree about whether the LCA’s copy survives. But such a tuple can never arise: a tag in a canonical LCA state is in *both* branches unless removed, and a removal is itself an event of the branch. The failing tuples are *infeasible*, and this is a theorem-grade phenomenon, not an inconvenience:

Finding. *For LCA-sensitive MRDTs beyond the group $\oplus$ lattice classes, the merge-coherence laws are irreducibly feasibility-conditioned: true on canonical tuples at honest LCAs, false in general. Quantifying merge VCs over all states — the published catalogue’s style — is not merely inconvenient for such data types; for several laws it is wrong in principle. (The enable-wins flag even falsifies the unit law $\text{mergeL}(\sigma_0, \sigma_0, s) = s$ on an infeasible state with a set flag and a zero counter.)*

Accordingly the delta laws enter the final contract in feasibility-conditioned form (**FeasibleDeltaVCs3**, displayed as Flat VCs 5–7): quantified over canonical states of weakly closed event sets, with every **mergeL** node at its honest LCA. The unconditional form remains available as a one-line *on-ramp* for the group and lattice classes — it implies the conditioned form outright (**feasibleDeltaVCs3_of_delta**).

7 The arbitration contract

RA-linearizability is defined against an arbitration (Definition 3.4), with `lo` — the order built from `rc` — the canonical one. What must an arbitration satisfy for a data type to be *provably* RA-linearizable against it? That contract is this section. The eight equations of §9 are then one *recipe* for meeting it, the recipe that builds `lo` from `rc`; a data type with a different arbitration (LWW’s timestamp order, below) meets the contract directly and skips them.

What the proof consumes. Trace `rc` through the adequacy argument (§10): it appears only through `loE`, and `loE` appears only as the arbitration, a per-event-set order. Nothing downstream inspects how that order was built. So the contract is a property of an abstract arbitration, written `arb C E` to make the configuration `C` explicit for the adequacy induction: the canonical state of `E` is the fold along any `arb C E`-respecting enumeration (`IsCanonicalStateArb`), RA-linearizability is per-version canonicity against it (`IsRALinearizable3Arb`), and it has six clauses.

Definition 7.1 (arbitration family, flat). *An arbitration family $\mathcal{F}$ supplies `arb` with six clauses, each a property the adequacy induction consumes at a named step:*

1. *extends-vis: every visible non-commuting pair inside E is ordered — $a, b \in E$, $a \xrightarrow{\text{vis}} b$, $a \not\Rightarrow b \Rightarrow \text{arb } C \ E \ a \ b$;*
2. *acyclic: `arb C E` has no cycle (maximal events exist for the peel), gated on $\xrightarrow{\text{vis}}$ being a strict partial order at the reachable C ;*
3. *antitone: shrinking the carrier never flips an edge — $E' \subseteq E''$, $\text{arb } C \ E'' \ a \ b \Rightarrow \text{arb } C \ E' \ a \ b$;*
4. *vis-consistent: `arb` never orders an event before one it observed — $b \xrightarrow{\text{vis}} a \Rightarrow \neg \text{arb } C \ E \ a \ b$, on any pair;*
5. *convergent: all `arb C E`-respecting enumerations of E fold to one state (`ArbConvergence`);*
6. *vis-local: `arb C E` reads only $\xrightarrow{\text{vis}}$ restricted to E — two configurations agreeing on $\xrightarrow{\text{vis}}$ over E agree on `arb` — E .*

ArbAdequacyReach.ArbFamily

Three of the six are forced by the re-attach and congruence steps, not by the surface design. The definition audit (§8) named acyclic, extends-vis, and convergence. Re-threading the adequacy induction over an abstract `arb` surfaced *antitone* (the re-attach step `isCanonicalStateArb_snoc` needs a maximal event to stay maximal on the smaller carrier), *vis-consistent* (an `arb` ordering a fresh event before a *commuting* predecessor stays acyclic and convergent yet breaks apply-maximality), and *vis-local* (the `createReplica` and `merge` congruence steps need `arb` blind to concurrent context outside E). Each is a genuine design requirement, and both instances below satisfy all six; but the theorem is mis-stated without them.

Theorem 7.2 (adequacy over an arbitration, flat). *Let $\mathcal{F}$ be an arbitration family and let the datatype supply the merge content against $\mathcal{F}$ ’s order — merge symmetry, and the feasible-delta and causal-delta equations of §9 read with `lo` replaced by `arb` (`FeasibleDeltaVCs3Arb`,*

CDVC3Arb). Then every reachable configuration is RA-linearizable per version against *arb*:
`IsRALinearizable3Arb(C, arb C)`. *ra_linearizable3Arb_of_core_feasible_cd*

The order clauses and the merge content are the two halves of adequacy, cleanly separated: the clauses make *arb* a usable linearization order (maximal events, one fold, stable across the transitions); the merge content makes *mergeL* land on that fold. *lo* is absent from both the statement and the proof — the fold is pinned solely by clause 5, convergence — so the engine is genuinely order-agnostic.

The conditioned family: one clause. Over the observational quotient (§13: *Inv*-guarded states, $\approx$ for $=$) the picture inverts. The conditioned engine does not *build* its Join from the delta laws; it takes the $\approx$ -Join as a primitive obligation (VC 4). So the conditioned arbitration family carries exactly *one* genuine clause, the $\approx$ -Join against *arb*, and the other five hold *for free*: extends-vis and vis-consistency are *arb*-independent (the visibility arm of $lo_{\approx}$ decides them), antitone is *loOnArb_mono* for any *arb*, and acyclic/convergent are not levers on a Join that is assumed rather than derived.

Theorem 7.3 (adequacy over an arbitration, conditioned). *Let $\mathcal{F}$ supply *arb* and the $\approx$ -Join against it (*EqJoinLemma3C_ArbH*), with the honesty and discipline conditions of §15. Then every reachable *Inv*-configuration is RA-linearizable per version against *arb*, up to $\approx$.*
`ra_linearizable3ArbEq_of_reach`

Finding (six versus one). *The flat family needs six order clauses; the conditioned family needs one. The asymmetry is structural, not incidental. The flat engine derives its Join from the delta and causal-delta VCs, and each of the six clauses is a lever in that derivation; the conditioned engine is handed its Join as VC 4, so the order clauses that built it fall away. Flat pays six clauses to construct the merge fact; conditioned pays one because it assumes it.*

Completeness: the contract is forced, on reachable states. The arbitration contract is not only sufficient but, on the states that arise, necessary. Reading the adequacy hypotheses off a datatype already known RA-linearizable recovers them.

Theorem 7.4 (converse, flat). *If a flat datatype’s canonical merge is a fold and converges (the content of RA-linearizability), then it satisfies the four-law core of §9 — feasible unit, the two feasible-redistribute laws, and the causal-delta equation — on every reachable, causally closed event set.*
Converse.converse_core

The converse is deliberately reachability-relative, and it is not a biconditional. RA-linearizability constrains the merge only on the tuples an execution produces; over abstract tuples the contract says strictly more, and no datatype is forced to the *specific* arbitration *lo*. The RESET datatype (write/write/reset) is RA-linearizable yet falsifies the conditioned swap witness VC 2 (*eqswap_not_forced*), and its invariant discipline VC 1 is a datatype choice, not a consequence (*vc_disc_extra*). The arbitration recast is thus a *sound abstraction of the sufficient condition*, exact on reachable states, not a characterization of RA-linearizability itself.

lo is one arbitration. The standard order lo^E is **arb** instantiated at **rc**: its non-commuting-visibility arm is **extends-vis**, its concurrent arm consults **rc**, and its absorber clause is what supplies antitonicity (**lo0n_isArb**, at **arb** := (**rc** = **Fst_then_snd**)). The next section is that instance — the eight VCs are the clauses of Definition 7.1 and the merge content of Theorem 7.2, spelled out for **lo**. A datatype with a different arbitration — the LWW register orders by timestamp, an order **rc** cannot express — skips **rc** and the eight VCs entirely and discharges the six clauses directly (**lwwFamily**); there **vis**-consistency rests on the Lamport-clock invariant, not on **rc**.

Worked example: the LWW register by timestamp. The recast is not idle generality. The last-writer-wins register is a datatype whose arbitration **rc** *cannot* express, and it routes through the adequacy engine unchanged.

The datatype. A state is $\perp$ (unset) or a winning write, a lex-ordered triple $w = (\text{time}, \text{replica}, \text{value})$ (**LWWWrite**). Update installs a write by semilattice **max**, **do**(s, e) = **max**(s, w_e); the three-way merge drops its LCA slot, **mergeL**(l, a, b) = **max**(a, b), since the state only ever moves up. The conflict resolver is trivial, **rc** = **Either** (**LWW_rc_either**).

lo *degenerates to nothing*. With **rc** = **Either** and all writes commuting (**max** is commutative, **LWW_all_comm**), *both* arms of lo^E are dead: the visibility arm needs a non-commuting pair, the concurrent arm needs **rc** = **Fst_then_snd**. So lo^E is the *empty* relation on LWW (**lww_lo0n_false**); the **rc** recipe produces the discrete arbitration, no order at all. LWW’s arbitration does not live in **rc** — it lives in the payload, the **max** on lex-timestamps.

The timestamp arbitration. Take **arb** $E a b := w_a < w_b$, the total order on writes by timestamp (**lwwArb**). It discharges the six clauses of Definition 7.1, and the discharge reads off where each clause’s content sits (**lwwFamily**): *acyclic* is strictness of $<$; *extends-vis* is vacuous (no non-commuting pairs); *antitone* and *vis-local* are immediate because **arb** ignores the carrier E entirely; *convergent* is the **max**-fold being order-independent; and *vis-consistent* is the one clause with content — $\xrightarrow{\text{vis}} b a$ forces **time**(b) < **time**(a) by the Lamport clock (**Configuration.causal_mono**), hence $w_b < w_a$, so **arb** never orders a write ahead of one it observed. That clause rests on the generation clock, *not* on the apply rule (which enforces only timestamp freshness): exactly where “the timestamp order respects causality” comes from.

Theorem 7.5 (LWW through the arbitration engine). *Every reachable LWW configuration is RA-linearizable against the timestamp arbitration **arb**, through the same capstone (Theorem 7.2) that certifies **lo** — **lo** absent, the fold pinned by **arb**’s own convergence. **lww_isRALinearizable3Arb_ts_via_capstone***

LWW is the witness that the abstraction is load-bearing: a datatype the **rc**-based catalogue can certify only by making its order vanish is linearized, at full strength, by the arbitration contract. (LWW also carries its ordinary flat discharge, **lww_ra_linearizable3**, against that empty **lo** with **max**-convergence pinning the fold; the two agree, and the timestamp arbitration is the honest account of the order the register actually realizes.)

8 Auditing the published definition: four strikes and the absorber

Why is the linearization order really just an arbitration? The recast of §7 is the endpoint of an audit of the published RA-linearizability definition (Definition 3.4), which reads its order lo^E (Definition 3.3) as an *arbitration-acyclicity* requirement plus a fold quotient, four of its clauses exposed as devices for that content rather than parts of it. Three strikes are recorded elsewhere in this paper; the fourth, the absorber, is decided by machine.

- *Strike 1: witness extension is not part of it.* The binary specification asks a merged version’s witness to *extend* its sides’ witnesses; the defeater (Example 4.5) refutes that on a fully LCA-legal configuration, so Definition 3.4 asks only that each version admit *some* witness, independently. The merge layer is rebuilt on canonical states and a Join Lemma that never traffics in witness lists.
- *Strike 2: rc-directionality and chain-freedom are reachability-relative.* Adequacy needs only that lo order non-commuting reachable pairs and stay acyclic on reachable sets, the content the converse recovers (Theorem 7.4). rc -non-comm and no- rc -chain are sufficient devices for it; LWW (§7) chains its rc along a total order and is RA-linearizable anyway.
- *Strike 3: the all-states quantification is a conditioning completion.* The unconditional halves of the merge VCs are evaluated by adequacy only on canonical states of reachable sets; each has an RA-linearizable violator whose failure lives at an unreachable state. The off-domain content is the conditioning completion, not the specification.
- *Strike 4: the absorber.* The absorber clause *removes* edges. Does dropping it change the verdict? Decided below.

The absorber dichotomy. Write plain for lo with the absorber dropped, keeping every rc -resolved concurrent edge. plain has a superset of lo ’s edges, so its respecting enumerations are a subset, so plain-RA-lin *implies* absorber-RA-lin : the absorber order is the weaker property. A separator would be a datatype RA-linearizable under the absorber but not under plain , making the absorber load-bearing.

The separator exists, and the mechanism is acyclicity, not a fold disagreement. It is the canonical add-wins OR-set. Two replicas each add then remove the same element x and merge at the empty root: events $a = \text{add } x$, $r_a = \text{rem } x$ on one replica, b, r_b on the other. In the merged event set $\{a, r_a, b, r_b\}$ the plain order is

$$a \rightarrow r_a \text{ (vis)}, \quad r_a \rightarrow b \text{ (rc, add-wins)}, \quad b \rightarrow r_b \text{ (vis)}, \quad r_b \rightarrow a \text{ (rc)},$$

a cycle $a \rightarrow r_a \rightarrow b \rightarrow r_b \rightarrow a$ that no list extends: plain-RA-lin has *no witness at all*. The absorber removes both rc edges (each add is absorbed by its own following remove), leaving the two vis chains; every interleaving folds to the empty live set, the operational merge state. The cycle alternates rc and vis edges, so its two rc edges are not consecutive and no- rc -chain (Flat VC 2) does not forbid it: the absorber is the clause that supplies acyclicity precisely when rc opposes visibility (add-wins orders a remove before a concurrent add, visibility orders an add before its own remove, and the two close a cycle). Overwrite datatypes never trigger it — LWW’s timestamp rc agrees with visibility, so no plain cycle and no separator.

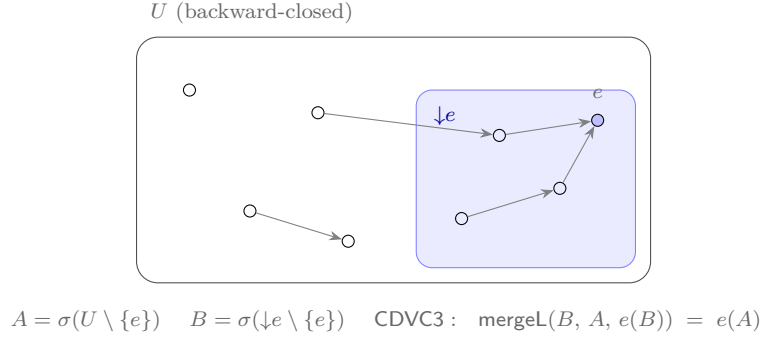


Figure 3: The causal-delta equation. The shaded region is e 's causal past $\downarrow e$ (what e saw); e is lo^U -maximal. e 's effect on the whole world A equals: compute e 's effect on its own causal past B , then merge that delta into A over LCA B . The merge sits at its honest LCA: $(U \setminus \{e\}) \cap \downarrow e = \downarrow e \setminus \{e\}$. Concurrent context that e never observed cannot amplify what e does.

The separation is certified and uniform. The exhaustive two-state, two-op-kind synthesis space (1408 specs) yields, among datatypes the metatheory certifies, **nine configuration-level separators, all via the cyclic route, none via a fold-value disagreement** (`absorber_dichotomy_check.py`, PASS+FAIL). Fold-value separator *versions* do occur, but only inside datatypes already non-RA-linearizable under the absorber order (the visible-swap specs `cond-comm` exists to exclude), so they never lift to a certified configuration. Plain-RA-lin on the certified route is *vacuous* (a cyclic relation has no linear extension), which is strictly stronger than an observable disagreement and not recoverable by coarsening $\approx$: coarsening the observation cannot rescue an order with no linear extension.

The invariant content. The four strikes converge on one reading, the one §7 makes primary: RA-linearizability asserts that a version's state is, up to $\approx$, the fold of some enumeration extending an *acyclic arbitration that orders every visible non-commuting pair by visibility*. The `rc` arm is one way to arbitrate concurrent pairs; no-`rc`-chain and the absorber keep that arbitration acyclic (the absorber the necessary half whenever `rc` opposes visibility); the all-states content is the conditioning completion; witness extension is not part of it at all. That is exactly the arbitration contract of Definition 7.1, with `lo` its canonical instance — the audit is why the recast re-presents the published definition rather than replacing it.

9 Deriving `lo`: the eight verification conditions

These eight equations are the `lo` instance of the arbitration contract (§7): the update-side obligations of §5 and the merge-side laws of §6 are exactly what make `lo`, the order built from `rc`, an arbitration family (Definition 7.1) and supply its merge content (Theorem 7.2). Of the six order clauses only two are carried by the update VCs — no-`rc`-chain (Flat VC 2) gives acyclicity, `cond-comm` (Flat VC 3) gives convergence — while `extends-vis`, `antitone`, `vis-consistency`, and `vis-locality` are structural properties of `lo`'s definition (its visibility arm and its absorber), and `rc-non-comm` (Flat VC 1) calibrates the concurrent arm to commutation. Flat VCs 5–8 supply the merge content. Assembled, they are a *new* VC set, replacing the paper's catalogue of twenty-

four. For an MRDT $\langle \Sigma, \sigma_0, \text{do}, \text{mergeL}, \text{rc} \rangle$, the contract is the eight conditions below. In the mechanization they are carried by three bundles — **CoreVCs3CD** (Flat VCs 1–4: the update triple **UpdateVCs** plus merge symmetry), **FeasibleDeltaVCs3** (Flat VCs 5–7), and **CDVC3** (Flat VC 8) — and the canonical route is $\text{CoreVCs3CD} + \text{FeasibleDeltaVCs3} + \text{CDVC3} \Rightarrow \text{Join Lemma} \Rightarrow \text{RA-lin}$.²

Update layer (unconditional, SMT-shaped). Write $o_1 \bowtie o_2$ for a *conflict-eligible* pair: distinct (**distinctOps**) and cross-replica (**differentReplicas**). Only conflict-eligible pairs can race, so this guard sits on every update-layer VC below.

Flat VC 1 (rc-non-comm).

$$o_1 \bowtie o_2 \implies (o_1 \not\stackrel{\text{rc}}{\rightarrow} o_2 \iff o_1 \stackrel{\text{rc}}{\rightarrow} o_2 \vee o_2 \stackrel{\text{rc}}{\rightarrow} o_1).$$

UpdateVCs.rc_non_comm_directional

The conflict table is the commutativity table, with a direction. Both guards are semantic content. The different-replica guard is a necessity, not a convenience: same-replica events are totally ordered by program order, so they are never concurrent and there is no conflict for **rc** to resolve. An unguarded VC would instead demand that **rc** order *every* non-commuting pair — unsatisfiable for real data types: the *optimized OR-set*’s same-replica same-element adds do not commute (the newer add evicts the older tag), yet they can never race, and its **rc** rightly ignores them. (The paper’s F^* artifact carries exactly this guard in its interface — **App_mrdt.fsti**: `requires distinct_ops o1 o2 /\ get rid o1 <> get rid o2`.)

Flat VC 2 (no-rc-chain).

$$o_1 \neq o_2 \wedge o_2 \neq o_3 \implies \neg (o_1 \stackrel{\text{rc}}{\rightarrow} o_2 \wedge o_2 \stackrel{\text{rc}}{\rightarrow} o_3).$$

UpdateVCs.no_rc_chain

Conflict priorities do not chain, which makes lo^E acyclic, so maximal events exist for the peel-and-recurse to select.

Flat VC 3 (cond-comm). For every state σ , events e, e', e'' , and event list π : if e, e', e'' are pairwise distinct, $e \stackrel{\text{rc}}{\rightarrow} e'$, and $e' \not\stackrel{\text{rc}}{\rightarrow} e''$, then

$$e''(\text{fold}(e(e'(\sigma)), \pi)) = e''(\text{fold}(e'(e(\sigma)), \pi)).$$

UpdateVCs.cond_comm_lift

A resolved conflict is invisible once an absorber runs: swapping an **rc**-ordered concurrent pair deep in a fold changes nothing for any later non-commuting event e'' , across arbitrary intervening events (the π). This is the engine of convergence.

²A historical wider bundle **CoreVCs3** adds three laws (**mergeL_init**, **lem_0op3**, **merge_peel_comm3**) that the causal-delta route never consumes; it is not part of the contract.

Merge layer.

Flat VC 4 (merge symmetry). $\text{mergel}(l, a, b) = \text{mergel}(l, b, a)$ for all states l, a, b .
CoreVCs3CD.mergeL_comm

Flat VC 5 (feasible unit).

$$E \subseteq E(C) \wedge \text{Can}_C(E, s) \implies \text{mergel}(\sigma_0, \sigma_0, s) = s.$$

FeasibleDeltaVCs3.feasible_init

Merging over nothing is a no-op — on states that can arise (the unconditional form is false for the enable-wins flag, the Finding of §6). The next two are the delta laws of Definition 6.1 on feasible tuples: every canonicity hypothesis pins one component to its history, and every *mergel* node sits at its honest LCA.

Flat VC 6 (feasible local-redistribute). Let $\xrightarrow{\text{vis}}$ be transitive and irreflexive on C ; let $E_1, E_2 \subseteq E(C)$ be weakly closed in C (Definition 5.3); let $e \in E_1$, $e \notin E_2$, and let e be $\text{lo}^{E_1 \cup E_2}$ -maximal ($\forall x \in E_1 \cup E_2$, $x \neq e \implies \neg(e \text{lo}^{E_1 \cup E_2} x)$). Then

$$\begin{aligned} & \text{Can}_C(E_1 \cap E_2, s_0) \wedge \text{Can}_C(\downarrow e \setminus \{e\}, B) \\ & \wedge \text{Can}_C(E_1 \setminus \{e\}, t_1) \wedge \text{Can}_C(E_2, s_2) \implies \\ & \text{mergel}(s_0, \text{mergel}(B, t_1, e(B)), s_2) = \text{mergel}(B, \text{mergel}(s_0, t_1, s_2), e(B)). \end{aligned}$$

FeasibleDeltaVCs3.feasible_local_redistribute

Flat VC 7 (feasible redistribute). In the same context ($\xrightarrow{\text{vis}}$ transitive and irreflexive on C ; $E_1, E_2 \subseteq E(C)$ weakly closed in C), now with $e \in E_1$, $e \in E_2$, and e $\text{lo}^{E_1 \cup E_2}$ -maximal as above:

$$\begin{aligned} & \text{Can}_C((E_1 \cap E_2) \setminus \{e\}, t_0) \wedge \text{Can}_C(\downarrow e \setminus \{e\}, B) \\ & \wedge \text{Can}_C(E_1 \setminus \{e\}, t_1) \wedge \text{Can}_C(E_2 \setminus \{e\}, t_2) \implies \\ & \text{mergel}(\text{mergel}(B, t_0, e(B)), \text{mergel}(B, t_1, e(B)), \text{mergel}(B, t_2, e(B))) \\ & = \text{mergel}(B, \text{mergel}(t_0, t_1, t_2), e(B)). \end{aligned}$$

FeasibleDeltaVCs3.feasible_redistribute

Flat VC 8 (the causal-delta equation, CDVC3). Let $\xrightarrow{\text{vis}}$ be transitive and irreflexive on C ; let $U \subseteq E(C)$ be weakly closed in C ; let $e \in U$ be lo^U -maximal ($\forall x \in U$, $x \neq e \implies \neg(e \text{lo}^U x)$); and let $\text{Can}_C(U \setminus \{e\}, A)$ and $\text{Can}_C(\downarrow e \setminus \{e\}, B)$. Then

$$\text{mergel}(B, A, e(B)) = e(A).$$

CDVC3

What an event does is determined by what it saw (Figure 3): applying e to the whole world A equals computing e 's delta on its recorded past $\downarrow e \setminus \{e\}$ and merging it in over that past. The merge sits at its honest LCA: $(U \setminus \{e\}) \cap \downarrow e = \downarrow e \setminus \{e\}$.

What the delta contract buys: one peel, three cases. The three feasible-delta laws are the three cases of a single move the Join Lemma (§10) makes on repeat: peel the $\text{lo}^{E_1 \cup E_2}$ -maximal event e out of the ternary merge, so the merge loses one event and the induction recurses. Flat VC 8 (CDVC3) is the atom: an event's effect on the whole world is its delta $e(B)$, computed on its own past $B = \sigma(\downarrow e \setminus \{e\})$, merged back in over B (Figure 3). It ties an event's effect to *what it saw*, and it closes every peel, collapsing $\text{mergeL}(B, \text{rest}, e(B))$ to $e(\text{rest})$. Flat VC 6 peels e when it lies in one side: its delta, buried in that branch, slides out and up over the whole merge (Figure 4); because the delta is taken over the shared past B , the merge cannot alter it, so it commutes past. Flat VC 7 peels e when it lies in both sides and the LCA, so its delta sits in all three inputs; the LCA slot cancels the triple-count and the three e -images merge to a single $e(B)$ factored out (Figure 5), the ternary stand-in for idempotence.

Reconciling the defeater. Example 4.5, where the bottom-up route was stuck, peels cleanly, and the peel exercises all three delta laws. Write a state as (A, D) with live set $A \setminus D$ (Example 4.5). The final merge is $\text{mergeL}(\sigma(v_s), \text{head}_p, \text{head}_q)$ with LCA $\sigma(v_s) = (\{A_p, A_q\}, \emptyset)$ (live on both tags), $\text{head}_p = \sigma(E_1) = (\{A_p, A_q\}, \{A_p\})$ ($E_1 = \{A_p, R_p, A_q\}$, live A_q), and $\text{head}_q = \sigma(E_2) = (\{A_p, A_q\}, \{A_q\})$ ($E_2 = \{A_p, A_q, R_q\}$, live A_p); the merge unions the dead sets to $(\{A_p, A_q\}, \{A_p, A_q\})$, all dead, which is $\sigma(U)$ for $U = E_1 \cup E_2$. On U the order is $\text{lo}^U = \{A_p \rightarrow R_p, A_q \rightarrow R_q\}$: the rc-arm would order each remove before the concurrent add (add-wins), but each add is overwritten by its own later remove, so the absorber drops both cross-edges, and the maximal events are the removes R_p, R_q .

- *Peel R_q (Flat VC 6, Figure 4).* R_q lies in E_2 only; its causal past is $\downarrow R_q = \{A_q\}$, so $B = \sigma(\{A_q\}) = (\{A_q\}, \emptyset)$ and its delta $R_q(B) = (\{A_q\}, \{A_q\})$. By merge symmetry the final merge is $\text{mergeL}(\sigma(E_1 \cap E_2), \sigma(E_2), \sigma(E_1))$, and VC 6 at $e = R_q$ slides R_q 's delta out of the $\sigma(E_2)$ branch onto the enclosing LCA slot:

$$\begin{aligned} & \text{mergeL}(\sigma(E_1 \cap E_2), \sigma(E_2), \sigma(E_1)) \\ &= \text{mergeL}(B, \text{mergeL}(\sigma(E_1 \cap E_2), \sigma(E_2 \setminus R_q), \sigma(E_1)), R_q(B)). \end{aligned}$$

The inner merge is over $U \setminus \{R_q\} = \{A_p, R_p, A_q\}$, and VC 8 collapses $\text{mergeL}(B, \sigma(U \setminus R_q), R_q(B))$ to $R_q(\sigma(U \setminus R_q))$. So R_q is applied to $\sigma(\{A_p, R_p, A_q\})$ (live A_q) over the past $\{A_q\}$ it saw: it kills A_q , and the concurrent A_p it never saw is untouched.

- *Peel A_q (Flat VC 7, Figure 5).* In $\sigma(U \setminus R_q) = \sigma(\{A_p, R_p, A_q\})$ the maximal event is now A_q : with R_q dropped from the set, A_q has no later remove to absorb it (antitonicity of lo , Definition 3.3). A_q lies in the LCA and both branches of this smaller merge, so its delta ($\downarrow A_q = \emptyset$, hence $B' = \sigma_0$ and $A_q(B') = (\{A_q\}, \emptyset)$) sits in all three inputs, and VC 7 factors the three copies out once:

$$\begin{aligned} & \text{mergeL}(\text{mergeL}(B', t_0, A_q(B')), \text{mergeL}(B', t_1, A_q(B')), \text{mergeL}(B', t_2, A_q(B'))) \\ &= \text{mergeL}(B', \text{mergeL}(t_0, t_1, t_2), A_q(B')), \end{aligned}$$

which VC 8 closes to $A_q(\sigma(\{A_p, R_p\}))$.

- *Peel R_p , then A_p .* $\sigma(\{A_p, R_p\})$ peels its maximal R_p by VC 6/8 to $R_p(\sigma(\{A_p\}))$ (over $\downarrow R_p = \{A_p\}$), and $\sigma(\{A_p\}) = A_p(\sigma_0)$ is the base. Composing the four peels,

$$\sigma(U) = R_q(A_q(R_p(A_p(\sigma_0)))) = \text{fold}(\sigma_0, [A_p, R_p, A_q, R_q]),$$

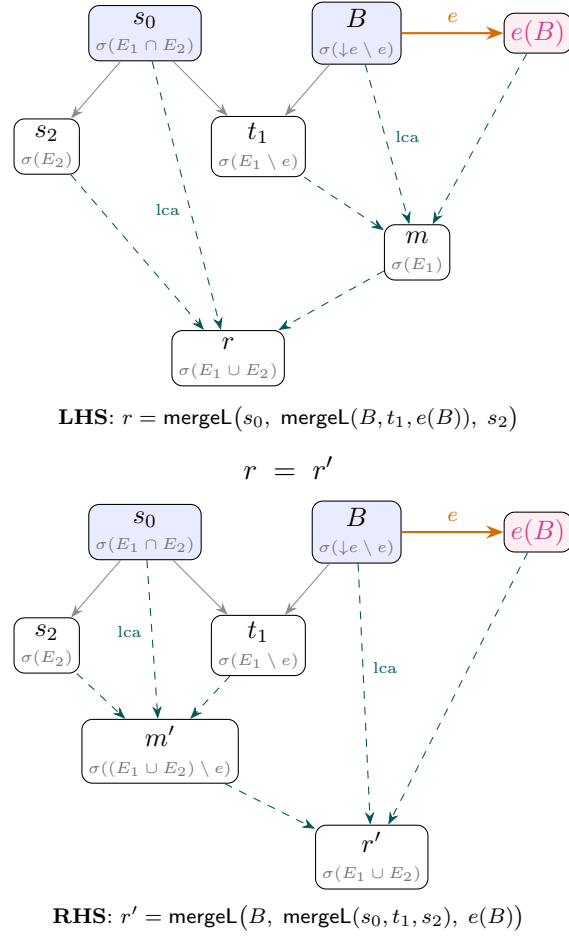
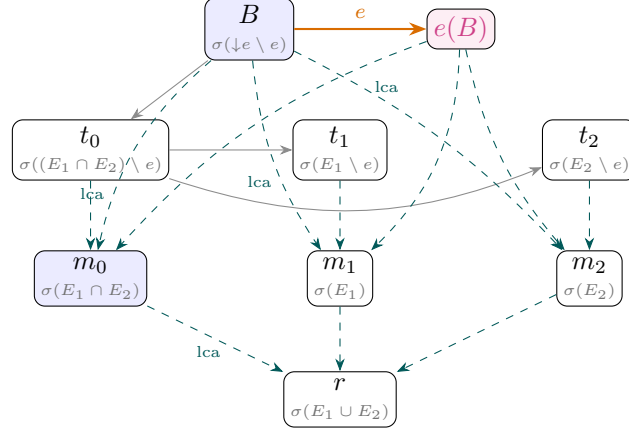
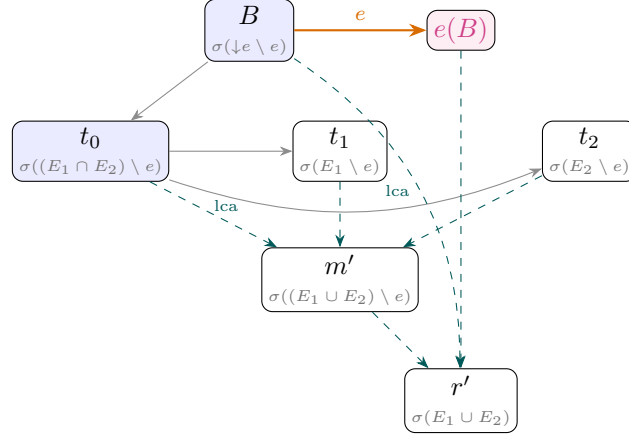


Figure 4: Flat VC 6 (VC6) as two merge arrangements over the same causal history, related by $r = r'$. Solid gray arrows are causal descent (the target's event set extends the source's); the orange arrow applies the peeled lo-maximal event e to the state of its own past, B , yielding its delta $e(B)$; dashed arrows feed a ternary `mergeL`, *lca* marks the LCA slot, and shaded states are the ones serving as LCAs. Every state is annotated with the canonical state $\sigma(\cdot)$ it realizes. **Top (LHS):** the delta enters nested inside the E_1 branch, rebuilding $m = \sigma(E_1)$ at LCA B ; the branches then merge at LCA s_0 . **Bottom (RHS):** the e -free world merges first at LCA s_0 into m' , and the same delta applies once on top, at LCA B . The VC is the equation $r = r'$: both realize $\sigma(E_1 \cup E_2)$.



LHS: $r = \text{mergeL}(\text{mergeL}(B, t_0, e(B)), \text{mergeL}(B, t_1, e(B)), \text{mergeL}(B, t_2, e(B)))$

$$r = r'$$



RHS: $r' = \text{mergeL}(B, \text{mergeL}(t_0, t_1, t_2), e(B))$

Figure 5: Flat VC 7 (VC7), conventions as in Figure 4. Here the lo-maximal event e lies in the LCA and both branches. **Top (LHS):** its delta $e(B)$ enters all three inputs of the Join-Lemma merge, each m_i rebuilt at LCA B from the e -free t_i ; the three then merge at LCA m_0 . **Bottom (RHS):** the e -free world merges first at LCA t_0 into m' , and a single copy of the delta applies on top, at LCA B : the triple count cancels, the ternary stand-in for idempotence. The VC is the equation $r = r'$: both realize $\sigma(E_1 \cup E_2)$.

the all-dead state: $\sigma_0 \rightarrow A_p$ (live A_p) $\rightarrow R_p$ (all dead) $\rightarrow A_q$ (live A_q) $\rightarrow R_q$ (all dead).

The order the peel produces is exactly the witness $[A_p, R_p, A_q, R_q]$ of Proposition 4.6, but reached without ever restricting it to a side. VC 8 ties each remove to its own past (R_q to $\{A_q\}$, R_p to $\{A_p\}$), so a remove kills only what it saw and the concurrent add it never saw survives by add-wins. The bottom-up route failed because it read that witness backwards off head_q , where the restriction $[A_p, A_q, R_q]$ runs R_q after A_p and folds to all-dead, not head_q 's live- A_p state. The delta contract never restricts: it peels the union's maximal event through the merge, and CDVC3 supplies the causal-past semantics the flat list cannot.

Flat VCs 1–4 are one-liners for every data type we have discharged; Flat VCs 6–7 frequently hold unconditionally anyway (for the OR-set, REDISTRIBUTE is a pointwise Boolean tautology); essentially the entire per-data-type proof effort concentrates in Flat VC 8, whose discharge follows a uniform recipe: characterize $\sigma(E)$ (a “sandwich” invariant along canonical enumerations), then a *trichotomy* placing every event the maximal e interacts with either inside $\downarrow e$ or absorbed on a side that owns it.

Compare the published catalogue: 24 VCs per data type, plus five more the soundness proof uses silently. Seventeen of the 24 — the BOTTOMUP induction-scheme families — exist to drive the broken induction and are consumed nowhere in the corrected one; the paper's MERGEIDEMPOTENCE is likewise never used. The seventeen were, however, doing honest work of a different kind: they are an *automation layer*, Skolemizing “holds on feasible states” into base-and-step equations an SMT solver can discharge. Rebuilding that layer, aimed at the corrected target (CDVC3 rather than BOTTOMUP), is in our view the most valuable open engineering problem this work leaves (§19).

10 Adequacy

The contract delivers: a data type owes exactly the eight equations of §9.

Theorem 10.1 (soundness of the eight VCs). *Let $\mathcal{D}$ satisfy Flat VCs 1–8 — the bundles CoreVCs3CD $\mathcal{D}$, FeasibleDeltaVCs3 $\mathcal{D}$, CDVC3 $\mathcal{D}$ — and let $\text{Inv}(\sigma_0)$ hold (for flat data types, trivially). Then every configuration C reachable from the initial configuration in the ternary transition system (Definition 3.2) is RA-linearizable, per version (Definition 3.4):*

lsRALinearizable3(C). $\text{ra_linearizable_of_core_feasible_cd3}$

The proof shape: the update layer powers the set-relative convergence theorem, hence canonical states; the transition induction maintains a five-clause invariant:

Definition 10.2 (good configuration). *GoodConfig3(C) demands:*

1. *canonical: every registered version is canonical for its event set — $\text{ver}(v) = (s, E)$ implies $\text{Can}_C(E, s)$;*
2. *vis-transitivity and*
3. *vis-irreflexivity of C 's visibility;*
4. *universe: registered event sets sit inside the observed universe — $\text{ver}(v) = (s, E)$ implies $E \subseteq E(C)$;*

5. *causal closure: registered event sets are fully $\xrightarrow{\text{vis}}$ -closed* — $\text{ver}(v) = (s, E)$, $a \xrightarrow{\text{vis}} b$, $b \in E$ imply $a \in E$.

Clause 1 delivers per-version RA-linearizability outright (*isRALinearizable3_of_good*); clause 5 is what makes full closure available to data types that need it — the asymmetry below. *GoodConfig3*

In the induction, fork and query are trivial and apply is the extend lemma; the merge case (*goodConfig3_merge_at*) reduces, via Lemma 4.2, to the ternary Join Lemma, which is proved from Flat VCs 4–8 by strong induction on $|E_1 \cup E_2|$ (*join_lemma3_of_cd_feasible*): select a $\text{lo}^{E_1 \cup E_2}$ -maximal e , decompose each side containing e through $\downarrow e$ (LOCAL-REDISTRIBUTE at the honest LCA $\downarrow e \setminus \{e\}$), cancel the shared delta (REDISTRIBUTE), close the principal step with CDVC3, and recurse; the join-to-RA-lin bridge is *ra_linearizable3_of_join*. The buried-event pathology of Example 4.5 dissolves structurally: no step ever asks how a *side*’s state factors — every factoring happens through the join.

Everything else one might fear owing is proved once, for all data types, as an execution-model invariant: visibility transitivity, per-version backward closure, store well-formedness, and Lemma 4.2 itself. The data type owes exactly the eight equations.

A second route, and an honest asymmetry. One discharged data type does not fit the pattern above. The enable-wins flag’s merge *compares counters* against the LCA, and for such merges the Join Lemma’s hypotheses as stated (backward closure under non-commuting-visibility) are provably too weak: there is a closure-legal tuple — a live LCA enable, a dead post-LCA enable inflating one side’s counter, the killing disable visible only to the other side — on which the production merge computes the wrong flag. Full *causal* closure (which the store invariant actually supplies) excludes it, and the flag is discharged through a full-closure variant of the join. The finding: **required closure strength is a function of the merge’s shape** — commutation-closure suffices for set-shaped merges, counter-comparison merges need full causal closure.

Definition 10.3 (full-closure and closure-indexed joins). *Call E fully closed in C when $a \xrightarrow{\text{vis}} b$ and $b \in E$ imply $a \in E$ (all visible predecessors, commuting or not; **fullClosure**); weak closure is the predicate of Definition 5.3 (**weakClosure**). *JoinLemma3F* is Definition 5.3 with both instances of premise (iii) strengthened from weakly to fully closed (**JoinLemma3F**). Generalizing, a closure predicate is any $\mathcal{C} : (C, E) \mapsto \text{Prop}$ reading only visibility and commutation (**ClosurePred**), and the closure-indexed join *JoinLemma3C* $\mathcal{D} \mathcal{C}$ is Definition 5.3 with premise (iii) replaced by $\mathcal{C}(C, E_1)$ and $\mathcal{C}(C, E_2)$. At $\mathcal{C} := \text{weak}$ (resp. **full**) closure it is definitionally the Join Lemma (resp. *JoinLemma3F*) — *joinLemma3C_weak*, *joinLemma3C_full* — and it is antitone in the index: strengthening the closure demanded of the sides weakens the lemma (**JoinLemma3C.anti**). *JoinLemma3C**

Reunifying the two routes might look routine — “full closure only strengthens what a data type may assume.” It is not: the naive reunification — re-run the Join Lemma induction with full-closure hypotheses — is refuted, because no single-event (or block) peel preserves full closure:

Proposition 10.4 (no proper block peel). *On the concrete four-event kill test — the set $U = \{e_{Ay}, e_{Rx}, e_{Ax}, e_{Ry}\}$ of the two-key datatype *K2* in its configuration C — no proper sub-block can*

be peeled from the back while preserving full closure: there is no $B \subseteq U$, nonempty and $\neq U$, that is upward closed under visibility within U ($a \in B, b \in U, a \xrightarrow{\text{vis}} b$ imply $b \in B$) and exits along no lo^U -edge ($e \in B, x \in U \setminus B$ imply $\neg(e \text{ lo}^U x)$). Dually, no proper down-closed front block with no entering lo^U -edge exists. **no_proper_back_block, no_proper_front_block**

What survives is the closure-indexed contract, in which the data type *declares* its closure strength; the soundness bridge is uniform across strengths, so no reunification into one strength is needed:

Theorem 10.5 (closure-indexed soundness bridge). *Let $\mathcal{C}$ be a closure predicate such that full closure implies it ($\forall C E, E$ fully closed in $C \Rightarrow \mathcal{C}(C, E)$ — the “declares its strength” side condition: store versions arrive fully closed, by Definition 10.2 clause 5), let $\mathcal{D}$ satisfy $\text{JoinLemma3C } \mathcal{D} \mathcal{C}$, and let $\text{Inv}(\sigma_0)$ hold. Then every configuration reachable from the initial configuration is RA-linearizable, per version.* **ra_linearizable3_of_joinC**

All nine production discharges route through this bridge unchanged — the file `Development/AllMRDTs_viaContract.lean` carries the nine `*_adequate_viaContract` theorems (OR-set, optimized OR-set, enable-wins flag, grow-only set and map, increment-only and PN counters, tombstone RGA, Peritext): eight at weak closure, the enable-wins flag at full.

11 The catalogue, discharged

Tables 1 and 2 show that production datatypes satisfy the VC set. The verification yields three lessons.

Tombstones simplify the metatheory. The catalogue’s structurally complex cases — the RGA sequence type and the Peritext rich-text type — land in the *easiest* tier. With tombstones, deletion *adds* a record; every state component is grow-only; all operations commute; the merge is a blind join. All of the RGA’s genuine difficulty lives in the *read-side* traversal (turning the node soup into a sequence), which RA-linearizability of the state never touches. By contrast, tombstone-free designs (which delete physically) fall outside the eight VCs for a reason worth stating precisely: their commutation VCs are themselves reachability-conditioned (two inserts commute only on well-formed states), and the update layer above, like the paper’s, quantifies over all states. The conditioned metatheory of §12 exists to host exactly this class, and the embedded-chain RGA of Part II is its canonical occupant, discharged through the factored route of §15.

Commutation class and delta class are independent axes. The multi-valued register is all-commuting, yet *not* in the unconditional tier: its merge is intersection-shaped, $(l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l)$, and fails the delta laws on infeasible tuples exactly as the OR-set does. The boundary between tiers is drawn by *how the merge uses its LCA*, not by whether operations commute. (The compensation: all-commuting makes every punctured causal past empty, $B = \sigma_0$, so the feasible discharge needs no trichotomy at all.)

The eight VCs certify real code on its historical fault line. The enable-wins flag’s repository history contains a known-broken sibling: a global-counter variant whose compositional

MRDT	contract tier	end-to-end theorem
Grow-only set σ : union of adds	unconditional	G0Set_ra_linearizable3_eq
Grow-only map σ : union of (k, v) records	unconditional	G0Map_ra_linearizable3_eq
Incr.-only counter σ : #events	unconditional	I0C_ra_linearizable3_eq
PN-counter σ : #inc – #dec	unconditional	PN_ra_linearizable3_eq
RGA (tombstone) σ : rgaApplicable: anchor closure, timestamp/id freshness, grave exclusion, and live removal; independent rgaSequentialSpec	guarded	rgaUnified
Peritext σ : ptApplicable: known/live character targets, known mark boundaries, and recorded operation-id freshness; removal timestamps are not stored	guarded	ptUnified
OR-set σ : tag live iff added, no vis-later rem	feasible	ORSet_ra_linearizable3_eq
OR-set (efficient) σ : ... or same-replica add later (eviction)	feasible	ORSetE_ra_linearizable3_eq
Enable-wins flag σ : per key: ctr = #enables; flag iff unkillable enable	full-closure	ewflagUnifiedF

Table 1: The Sal production catalogue, flat tier; each entry’s second line is its one-line σ -characterization. The flat data types conclude through the ONE generic conditioned metatheorem (§12–§14) at the identity instantiation ($\approx =$, their VC bundles feeding the flat collapse `FlatGeneric_Bridge.lean`, instance theorems in `MRDT_Instances/` (per-RDT)). The contract-tier column is each data type’s Join-Lemma discharge content; the historical flat corollaries over the raw system remain in the instance directories as internal steps.

VC (`inter_right_1op`) fails on DAG-shaped executions, the very defect per-replica counters were introduced to fix. The mechanized σ -facts prove the production merge computes exactly union-liveness — *verified on precisely the corner where the sibling fails*: the metatheory explains *why* the fixed design is right, not merely that it converges.

12 The conditioned metatheorem

The eight VCs of §9 quantify over *all* states, which §11 showed is fatal for state-dependent datatypes: a sequence with physical deletion has two concurrent operations commuting only on well-formed states at which both are applicable, so over all states the commutation VC is false. The remedy is the general signature of §2: condition every VC on `Inv` and `app`, relax the witness fold of Definition 3.4 to the observational equivalence $\approx$, and prove RA-linearizability from the conditioned VCs once. This section and the next two state the conditioned VCs and the metatheorem with its pen-and-paper proof; §15 then gives the factored discharge route the production instances use.

MRDT	contract tier	end-to-end theorem
Multi-valued register σ : tagged writes $\times$ overwrite log; all-comm $\Rightarrow B=\sigma_0$	guarded	mvrUnified
Add-wins priority queue σ : the OR-set pattern on A; grow-only I	feasible	AWPQ_ra_linearizable3_eq
Bounded counter (escrow) σ : public guard bcApplicable ; convergence plus BCInv at every version and bc_value_nonneg ; independent per-replica balance refinement	all-comm	boundedCounter_flagship boundedCounterSequentialRefinement
FWW reservation register σ : payload arbitration (min-ts semilattice); winner characterized at every version (fwv_version_min)	all-comm	FWW_ra_linearizable3_eq
LWW register σ : payload arbitration (max-ts semilattice), the last-writer twin of FWW; winner characterized (lww_version_max); <i>end-to-end mechanized</i> , unlike the flat CRDT LWW (sorry-carrying VC bridge)	all-comm	LWW_ra_linearizable3_eq
BudgetCart σ : budgeted cart, spend derived from live instances; safety <i>gated</i> on causal canonicity (OQ 19)	or-set rc	BCart_ra_linearizable3_eq
Mergeable queue (Peepul) σ : public existential mint guard qApplicable ; no rc exists (enqueue clique); the merge itself is the linearization witness	<i>direct join</i>	queue_ra_linearizable3
RGA (embedded-chain) σ : public guard eApplicable ; RA-lin at every honestly reachable configuration; canonical sorted state, fold-canonical (e_fold_canon); code-parametric (Part II)	<i>direct join</i>	embed_ra_linearizable3
RGA (sided, two-sided) σ : public guard sApplicable ; for every side assignment; the one-sided datatype is its all-R fragment (Part II)	<i>direct join</i>	sided_embed_ra_linearizable3
Peritext (embed kernel, fused) σ : one RGA at $\alpha := \text{char} \oplus \text{boundary}$; deleting a character never re-formats another (renderIds_del ; Parts II–III)	<i>instantiation</i>	peritextEmbed_ra_linearizable3

Table 2: The Sal production catalogue, conditioned and direct-join tiers (continuing Table 1). The direct-join instances conclude through the per-configuration join hook under honest reachability (§15); the conditioned instances carry their safety content as separate displayed theorems where noted.

13 The conditioned verification conditions

The conditioned signature $\langle \text{State}, \sigma_0, \text{do}, \text{merge}, \text{Inv}, \text{app} \rangle$ is richer than the flat tuple of §9: it adds **Inv** and **app**. The four VCs below are the general contract, and the eight flat VCs of §9 are their identity instantiation ($\text{Inv} = \text{app} = \top$, $\approx = =$), discharged through **FlatGeneric_Bridge**. The two are one ladder, not two catalogues: a flat datatype supplies the eight, a conditioned datatype the four, never all twelve. Like the eight, the four are an arbitration instance (§7): they make $\text{lo}_{\approx}$, the order from rc read up to $\approx$, the conditioned arbitration family of Theorem 7.3, whose one genuine clause is the $\approx$ -Join — VC 4 below — the other five holding for any arbitration.

A datatype presents a signature $\langle \text{State}, \sigma_0, \text{do}, \text{merge}, \text{Inv}, \text{app} \rangle$ where **Inv** is a predicate on states and **app** a predicate on (operation, state) pairs (Lean: **ConditionedMRDTSig**, **Framework/MRDTSig.lean**). It must supply:

VC 1 (Discipline). *Inv σ_0 holds; do preserves Inv on applicable operations; and generation is disciplined: every operation is app and fresh at its birth state, with globally unique, monotone ids. Formally, the execution model is the conditioned configuration: an event set $E(C)$ with a*

visibility relation $\xrightarrow{\text{vis}}$ subject to

- (i) irreflexivity, transitivity, and endpoint membership (both ends of every $\xrightarrow{\text{vis}}$ -edge are events);
- (ii) global timestamp uniqueness: distinct events of $E(C)$ carry distinct timestamps;
- (iii) Lamport monotonicity: $a \xrightarrow{\text{vis}} b$ implies $\text{time}(a) < \text{time}(b)$ (the generation clock increases along $\xrightarrow{\text{vis}}$);
- (iv) the two invariant clauses: $\text{Inv}(\sigma_0)$, and $\text{Inv}(\sigma) \wedge \text{app}(e, \sigma) \Rightarrow \text{Inv}(\text{do}(\sigma, e))$.

App-at-birth is deliberately not a structure field: it is carried by the honesty contracts of §15, which is where “fresh and applicable at its birth state” becomes consumable. (Kernel-clean, axiom-free.) `ConditionedConfiguration, Framework/ConditionedExecutionModel.lean`

VC 2 (Conditioned commutation — the swap witness). For concurrent operations a, b and any state σ with $\text{Inv } \sigma$ at which both are applicable in the relevant reorder sense, the swap witness holds:

$$\text{EqSwap}(a, b, \sigma) :\iff \text{do}(\text{do } \sigma \ a) \ b \approx \text{do}(\text{do } \sigma \ b) \ a.$$

The states σ at which the witness is owed are pinned by VC 3.

EqSwap

VC 3 (Threadable reorder invariant). An auxiliary invariant J on (state, pending-event) that (i) certifies applicability/the swap precondition of VC 2 at reorder states, (ii) holds at the enabling fold of each event, and (iii) is preserved by each reorder step. Only the enabled events — those whose causal past is folded — need be certified; that scope is exactly what a linearization repair ever transposes. Formally, VCs 2 and 3 are jointly the swap oracle of the convergence engine (Theorem 14.3): for a pending set E , a duplicate-free lo^E -respecting prefix list π drawn from E , and events $a, b \in E \setminus \pi$ with $a \neq b$, a and b lo^E -incomparable, and both enabled at π

$$(\forall z \in E. \ z \neq a \wedge z \text{ lo}^E a \Rightarrow z \in \pi, \text{ and likewise for } b),$$

the oracle must supply $\text{EqSwap}(a, b, \text{fold}(\sigma, \pi))$. The enabledness restriction is clause (ii)+(iii) made precise: the oracle is consulted only at prefixes containing both swapped events’ whole lo^E -past, exactly the prefixes a repair visits.

VC 4 (Merge-linearization bridge — the conditioned Join Lemma). For a reachable LCA state l and branches a, b obtained from l by disjoint concurrent event lists, $\text{mergel } a \ b \approx \text{fold } l \ \pi$ for a lo^E -respecting enumeration π of the branch events. Formally: $\text{at } \approx =$ this is exactly the per-configuration ternary Join Lemma of Definition 5.3 (*JoinLemma3At*). At a nontrivial quotient it is the $\approx$ -relaxed form (*EqJoinLemma3C, Metatheory/GenericEqQuotient.lean*): with canonicity up to $\approx$

$$\text{Can}_{\tilde{C}}(E, s) :\iff \exists \rho. \ \rho \text{ enumerates } E \wedge \rho \text{ respects } \text{lo}_{\approx}^E \wedge \text{fold}(\sigma_0, \rho) \approx s$$

(*IsCanonicalStateEq*; $\text{lo}_{\approx}^E$ is lo^E with commutation read as $\approx$ -commutation on Inv -states), the join demands: for Inv -satisfying s_0, s_1, s_2 and fully closed sides E_1, E_2 , each of which, and their union $E_1 \cup E_2$, satisfies the datatype’s generation-discipline predicate (a bundle parameter), canonicity of s_0, s_1, s_2 at $E_1 \cap E_2, E_1, E_2$ implies $\text{Can}_{\tilde{C}}(E_1 \cup E_2, \text{mergel}(s_0, s_1, s_2))$. The fold is the raw update: the quotient layer’s internal guard is discharged at the transfer, so the datatype states its $\approx$ -join over its own do, not a guarded surrogate.

14 The metatheorem

Theorem 14.1 (Soundness — conditioned VCs $\Rightarrow$ RA-linearizability (*pen and paper*)). *Any MRDT satisfying VCs 1–4 is RA-linearizable (Definition 3.4, read up to $\approx$).*

Status of the theorem. This general form (arbitrary `Inv`, `app`, $\approx$) is a paper-level statement, proved below by two paper-level lemmas; deliberately, no single Lean declaration carries it, and none is cited on it. What is mechanized is its *instances*, which are the capstones the catalogue actually rests on: the flat instance (`Inv = app =  $\top$ ,  $\approx = =$`) is Theorem 10.5 (`ra_linearizable3_of_joinC`); the convergence engine consumed by Lemma 14.2 is Theorem 14.3 (`eq_convergence`); and the $\approx$ -quotient layers (`Metatheory/GenericEqQuotient.lean`, `GenericEqQuotient_H.lean`, `GoodConfig3H.lean`, capstone shape `IsRALinearizable3Eq`) mechanize the adequacy induction at the quotient for the production instances of §15 and Part II.

The proof is generic — it never inspects a particular datatype — and factors through two lemmas, each proved over the abstract signature with $\approx$ an arbitrary congruence for `do` and `merge`.

Lemma 14.2 (Convergence). *VCs 1, 2, 3 imply that all lo^E -respecting admissible enumerations of a backward-closed reachable E fold from σ_0 to $\approx$ -equal states; hence $\sigma^*(E)$ is well defined up to $\approx$.*

Proof. Any two lo^E -respecting enumerations of E are related by repeatedly transposing adjacent lo^E -incomparable events. Because lo^E is non-transitive, “respecting” is the elementwise `Pairwise` condition rather than sortedness, so this order-repair is more delicate than for a total order — it is exactly what the generic engine `eq_convergence` carries out, and we appeal to it rather than re-derive it. Each transposition preserves the fold up to $\approx$ by VC 2, provided the swap precondition holds at the transposition point; VC 3 supplies it along the whole enumeration (base at each event’s enabling fold, preserved by each step, scoped to the enabled events — the only ones the repair transposes). The engine consumes only that $\approx$ is an equivalence, `do` is $\approx$ -congruent, and a swap oracle of the shape VC 2+3 provides (Lean: `eq_convergence`, `kernel-clean`). $\square$

The engine itself is a displayable theorem, and its statement is the precise form of the two VCs’ joint content:

Theorem 14.3 (the generic $\approx$ -convergence engine). *Fix an update `do` over a state space with an equivalence $\approx$ for which the fold is congruent, and any binary relation ℓ on events (the engine is order-agnostic; Lemma 14.2 instantiates $\ell := \text{lo}^E$). Let E be a finite event set and π_1, π_2 two enumerations of E , each respecting ℓ , and let σ be any start state. Suppose the swap oracle holds at σ : for every duplicate-free ℓ -respecting list π drawn from E and every pair $a, b \in E \setminus \pi$ with $a \neq b$, $\neg \ell(a, b)$, $\neg \ell(b, a)$, and both enabled at π (every ℓ -predecessor in E of a , and of b , lies in π), `EqSwap`(a, b , `fold`(σ, π)). Then*

$$\text{fold}(\sigma, \pi_1) \approx \text{fold}(\sigma, \pi_2).$$

eq_convergence

Remark 14.4 (where the engine lives, and what is generic in the mechanization). *The declaration's home is `MRDT_Instances/RGA_Rehoming/RGA_ConditionedConvergence.lean`: an instance directory, not `Framework/` or `Metatheory/`, a bookkeeping accident of its genesis in the rehoming stress test. Its statement is payload-generic (the element type is a parameter) and order-generic in ℓ , but it is stated over that file's concrete sequence-state functor with its concrete $\approx$ (proved an equivalence with congruent fold in the same file), not over an abstract signature; the fully abstract reading is exactly Lemma 14.2's pen-and-paper step. Two mechanization details folded out of the display: the Lean statement threads an explicit length parameter for the strong induction (universally quantified, hence eliminable), and the oracle restriction to enabled prefixes is load-bearing: quantifying π over all lists is unsatisfiable, since several swap-discharge conjuncts are provably false at junk prefixes.*

Lemma 14.5 (Canonical adequacy). *VC 4 and Lemma 14.2 imply that every reachable version state is $\approx \sigma^*$ of its delivered event set.*

Proof. Induction over the version DAG. Initial: the empty fold. Update node: appends one delivered operation to a canonical fold (definitional). Merge node with LCA l and branches a, b : by hypothesis $a \approx \sigma^*(E_a)$ and $b \approx \sigma^*(E_b)$, and $l \approx \sigma^*(E_l)$. First rewrite a, b to literal canonical folds using $\approx$ -congruence of `merge` (so VC 4, stated for branch folds, applies); then VC 4 gives `merge l a b` $\approx$ `fold l  $\pi$` for a lo^E -respecting π of the branch events $E_a \cup E_b$. Now E_l causally precedes those branch events (they were generated on states derived from l), so a lo^E -respecting enumeration of $E_l \cup E_a \cup E_b$ factors as (an lo^E -enumeration of E_l) followed by π ; hence `fold l  $\pi$` = `fold ( $\sigma^*(E_l)$ )  $\pi$` $\approx \sigma^*(E_l \cup E_a \cup E_b)$ by Lemma 14.2 (the engine is start-state generic). Backward closure keeps every enumeration admissible. $\square$

Proof of Theorem 14.1. Immediate from Lemma 14.5: each version state $\approx \sigma^*(E_v)$, which is Definition 3.4. (Unconditioned template: `ra_linearizable3_of_joinC` plus the closure-indexed adequacy bridge, already kernel-clean with nine instances; the conditioned metatheorem generalizes it by carrying `lnv/app` through the two lemmas.) $\square$

Corollary 14.6 (Strong eventual consistency). *Two versions with the same delivered set E have $\approx$ states: both $\approx \sigma^*(E)$ by Theorem 14.1, hence $\approx$ each other.*

Remark 14.7 (The flat instances). *The nine flat production MRDTs (counter, OR-set, MVR, ...) take `lnv = app = true`: VC 2 is their unconditioned commutation, VC 3 is `J = true`, and VC 4 is the ordinary Join Lemma — already discharged (`ra_linearizable3_of_joinC`). They inherit RA-linearizability from the same Theorem 14.1. The instances of §15 and Part II exercise the conditioned mechanisms proper.*

15 The factored discharge route: honest reachability and per-configuration joins

The conditioned instances in production do not each re-prove VCs 2 and 3 from scratch. What they share is an induction: thread a per-configuration honesty contract through reachability, and invoke a Join Lemma at each merge. This section presents that induction, extracted once, together with the instances that ride it: the mergeable queue (the flagship, whose conflict graph admits no `rc` at all), the bounded counter (conditioning for safety), and the FWW reservation register (payload arbitration). Part II's embedded-chain RGA is discharged by exactly this route.

The factorization: one metatheorem, three join species. The common structure across the conditioned instances is a theorem rather than a convention: the shared induction is extracted once (`Metatheory/HonestReach.lean`), as the next definition and theorem, and the queue’s and the bounded counter’s bespoke inductions are one-line corollaries of the extracted form.

Definition 15.1 (honest reachability). *Fix a per-configuration contract H and the hypothesis $\text{Inv}(\sigma_0)$ (displayed here and in every consumer below; flat data types discharge it trivially). $\text{HonestReach} \mathcal{D} H$ is the least predicate on configurations containing the initial configuration (Definition 3.2) and closed under: if C is honestly reachable, $H(C)$ holds, and $C \xrightarrow{\ell} C'$ is a Step3 transition, then C' is honestly reachable. The contract is checked at the pre-step configuration.*
HonestReach

Theorem 15.2 (the factored metatheorem). *Let H be a contract such that every H -configuration admits the Join at its core: for all C' , $H(C')$ implies $\text{JoinLemma3At} \mathcal{D} C'$ (Definition 5.3). Then, with $\text{Inv}(\sigma_0)$, every H -honestly reachable configuration is RA-linearizable, per version. Axioms: **propext**, **Quot.sound**. Plain reachability is the degenerate contract $H = \top$ (**honestReach_of_reachable**), recovering the unconditional bridge. **ra_linearizable3_of_honest_reach***

What does *not* factorize is the join discharge, and that is the finding: the instances discharge **JoinLemma3At** by genuinely different mechanisms — *conditioned commutation* (operations commute on invariant-satisfying states, the framework’s fully general quotiented route), *flat commutation* (the bounded counter: the CD route, contract $\top$), and *direct witness* (the queue: the merge is its own linearization certificate; Part II’s embedded-chain family discharges the same way). Conditioning thus splits as

generic honest-reachability metatheorem $\times$ per-datatype join discharge,

and the framework’s job at the boundary is to *receive* a join, not to produce one. A witness-classed generalization (**JoinLemma3AtW**, `Metatheory/WitnessClass.lean`, from the Shesha investigation of Part III) extends the hook to datatypes whose canonical states are unique only per witness-classed enumerations; and the instance layer is payload-generic where it should be (the embed instance of Part II takes its element type as a defaulted parameter, so the fused Peritext is a pure instantiation). One further regularity: the production honesty contracts all have the shape “ P holds of each event at the fold of its causal past” — anchor liveness for the sequence family, slack for the counter, head-ness for the queue — with P the signature’s client-checkable **app** (or its accuracy analogue). That shape is extracted once (`Metatheory/GenHonest.lean`):

Definition 15.3 (the generic honesty shape). *For a predicate P on (event, state) pairs,*

$$\begin{aligned} \text{GenHonest} \mathcal{D} P C &: \iff \forall e \in E(C). \forall \pi. \pi \text{ enumerates } \{e' \in E(C) : e' \xrightarrow{\text{vis}} e\} \\ &\implies P(e, \text{fold}(\sigma_0, \pi)), \end{aligned}$$

P of every event at the fold of every enumeration of its causal past. **AppHonest** $\mathcal{D} := \text{GenHonest} \mathcal{D} \text{app}$ is the canonical instance. The companion side condition **CausalPastEnumerable** (every event’s causal past admits an enumeration) holds in reachable configurations, whose event sets are finite, but is kept as an explicit hypothesis where consumed: the repository carries no generic finiteness result for reachable event sets. **GenHonest**, **AppHonest**

Remark 15.4 (two honesty names: $\forall$ versus $\exists$). *The mechanization carries two near-collision names, and both are load-bearing. `AppHonest` (`Metatheory/GenHonest.lean`) is the $\forall$ -form above: `app` at the fold of every enumeration of the causal past. `HonestApp` (`Metatheory/GenericSafety.lean`, Definition 15.12 below) is the $\exists$ -form: `app` at some causal fold of the causal past, the one the issuing replica materialized. The $\forall$ -form feeds convergence contracts, whose guards are fold-order-insensitive; the $\exists$ -form feeds the safety metatheorem, because for order-sensitive datatypes the $\forall$ -form is unsatisfiable once a past holds two surviving candidates (different enumerations materialize different heads).*

Theorem 15.5 (the composite generic-honesty metatheorem). *For any P : if every $(\text{GenHonest } D \ P)$ -configuration admits the Join at its core, then, with $\text{Inv}(\sigma_0)$, every configuration honestly reachable under the contract $\text{GenHonest } D \ P$ is RA-linearizable, per version.*
`ra_linearizable3_of_genHonest_reach`

The counter's and the queue's contracts are re-derived as instantiations (`BCHonest_iff_genHonest`, `qHonest_of_genHonest`). A generic *safety* metatheorem completes the factorization; it is the subject of a paragraph below, and the counter's counting argument indeed turned out to mark where the per-datatype content sits.

Direct-join instance: the mergeable queue — when no rc can exist. The queue of the certified-MRDT line of work (Soundarapandian et al., PLDI'22, *Certified Mergeable Replicated Data Types*) is the catalogue's first data type whose conflict graph is a *clique* rather than a star: two concurrent enqueues genuinely do not commute (queue order is arrival order), so VC 1's iff forces an rc-edge between them, and any orientation of a self-conflicting class composes with itself into a length-2 path — every candidate assignment violates `no_rc_chain` (Open Question 19). The flat engine is structurally unavailable, and the instance goes through the framework's per-configuration join hook instead (`JoinLemma3At`, `goodConfig3_merge_at`): prove the Join Lemma directly, at every reachable configuration.

The implementation is the functional queue with Peepul's merge, verbatim: the state is a list of $(\text{tag}, \text{value})$ pairs, head first; `enq v` appends a fresh element tagged with the operation's own timestamp; `deq t` removes the element tagged t , wherever it sits; and the three-way merge keeps the LCA's elements that survive in both branches (in LCA order), then each branch's new arrivals (in branch order). The client-facing API is unchanged — `dequeue()` at a replica reads its local head — but the *operation* records which element that call removed: `app` for `deq t` is “ t is the head of the issuing replica's state” ($\text{qApplicable}(\text{deq } t, \sigma) :\Leftrightarrow \exists v. \text{head}^?(\sigma) = (t, v)$; enqueues are unconditional). As with the sequence family, the generation discipline is the datatype's natural client behaviour, captured rather than imposed.

Definition 15.6 (the queue's honesty contract). $\text{QHonest}(C)$: *every dequeue names a tag its issuer had observed as an enqueue,*

$$\forall e \in E(C). \text{op}(e) = \text{deq } t \implies \exists a \in E(C). a \xrightarrow{\text{vis}} e \wedge \text{time}(a) = t \wedge \exists v. \text{op}(a) = \text{enq } v.$$

The same clause read on the replica-keyed core is `QHonestCore`; the ternary form transfers to it (`qHonest_core`).
`QHonest`

Theorem 15.7 (the queue's join, at every honest configuration). *At every core configuration satisfying `QHonest`, the ternary Join Lemma holds: `JoinLemma3At Q C`.*
`q_join_at`

Theorem 15.8 (the queue capstone). *Every configuration honestly reachable under QHonest (the instance QReach := HonestReach Q QHonest of Definition 15.1, with $\text{Inv}(\sigma_0)$ trivial) is RA-linearizable, per version.* *queue_ra_linearizable3*

The proof of Theorem 15.7 exhibits Peepul’s merge as its own certificate. The Join Lemma asks for a delivery-respecting enumeration of $E_1 \cup E_2$ whose fold is $\text{mergeL}(\sigma_0, \sigma_1, \sigma_2)$; the witness is the concatenation $\rho_0 \cdot (\rho_1 \setminus E_0) \cdot (\rho_2 \setminus E_0)$ of the given enumerations — exactly the merge’s shape, no reordering. Three facts make the fold of that concatenation compute the merge, all consequences of the closure premise (branch event sets are backward-closed under $\xrightarrow{\text{vis}}$ among non-commuting pairs), timestamp uniqueness, and the honesty contract (discharged by the **app** head-check via **qHonest_of_applicable**: applicability at *some* fold of the issuer’s causal past suffices, the $\exists$ -form of Remark 15.4): a fold from the empty queue is “the enqueues, in order, minus the dequeued tags” (the fold formula, which needs honesty — a dequeue preceding its enqueue would consume nothing); a delta’s tags are fresh with respect to the LCA (uniqueness); and a cross-branch dequeue is impossible — a **deq** t in one branch pulls its enqueue into that branch by closure, so if the enqueue of t is the *other* branch’s delta, it would lie in both branches, hence in the LCA, contradicting delta-ness. An LCA element then survives the union iff it survives in both branches, which is precisely the merge’s first clause.

The payoff is the specification. In the PLDI’22 development the queue was the hardest case exactly because its specification had to be written in a bespoke relational language over event partial orders, with an explicit clause recovering *which element is the head*; the merge was then verified against that ad-hoc spec. Here the specification is the same generic statement as every other instance in Table 1 — every version is the fold of a delivery-respecting linearization of its event set — and head-ness, FIFO order, and once-per-element consumption are *read off* the fold rather than specified. (One semantic caveat, present equally in Peepul: two replicas that concurrently dequeue the same head both consume that one element — the merged state removes it once. RA-linearizability makes this visible instead of hiding it: both **deq** t events fold over the single **enq** t .) At ~ 870 lines (**MRDT_Instances/MergeableQueue/**), the instance carries positional structure at a fraction of a sequence datatype’s cost.

Conditioning for safety: the bounded counter. Conditioning first entered the framework to deliver *convergence* for state-dependent datatypes. The bounded counter (per-replica escrow: grow-only tallies (**incs**, **decs**) per replica, per-slot group merge; mirror of **Sal/CRDTs/Bounded_Counter**) shows the same mechanisms delivering *safety* for a datatype whose convergence is flat: all its operations commute, the eight VCs discharge along the counter route, and the catalogue capstone (**BC_ra_linearizable3_eq**) is an identity instantiation. What the flat theory cannot even state is the property the datatype is named after. Its CRDT development is explicit: “the bound is enforced operationally by well-behaved clients” — a client refuses to issue **Dec** without slack — and that discipline lives outside the formal development. Conditioned, it moves inside: **Inv** is the per-replica escrow invariant ($0 \leq \text{decs } r \leq \text{incs } r$), **app** is the client’s check (a **Dec** needs slack in the *issuing replica’s own* slots — positive, and checkable against the issuing replica’s state), and the contract on executions is the counter’s honest-delivery contract:

Definition 15.9 (the counter’s honesty contract). **BCHonest**(C): *every decrement was applicable*

at the fold of its causal past,

$$\forall e \in E(C). \text{ op}(e) = \text{dec} \implies \forall \pi. \pi \text{ enumerates } \{e' \in E(C) : e' \xrightarrow{\text{vis}} e\} \implies \text{app}(e, \text{fold}(\sigma_0, \pi)),$$

with $\text{app}(\text{dec by } r, \sigma)$ the issuer's own-slot slack check $\text{decs}_r(\sigma) + 1 \leq \text{incs}_r(\sigma)$. This is exactly the generic shape of Definition 15.3 at $P := \text{app}$: the dec-guard is immaterial because app is $\top$ on increments (*BCHonest_iff_genHonest*). *BCHonest*

Theorem 15.10 (the bound, per version). *In every configuration reachable in the plain LTS (from the initial configuration, $\text{Inv}(\sigma_0)$ trivial) whose history satisfies *BCHonest*, every registered version satisfies the escrow invariant: $\text{ver}(v) = (s, E)$ implies $0 \leq \text{decs}_r(s) \leq \text{incs}_r(s)$ for every replica r (heads, LCAs, everything the store registered). Hence, for every finite replica list rs , the readable value is non-negative: $0 \leq \sum_{r \in rs} (\text{incs}_r(s) - \text{decs}_r(s))$ (*bc_value_nonneg*). *bc_version_inv**

The original proof is a counting argument riding the flat machinery: every version's state is a fold of its event set (canonicity), fold slots are per-replica event counts (the group merge is inclusion–exclusion on counts, via the LCA lemma), and the vis-maximal decrement's honesty at its own causal past — which lies inside the version by causal closure — bounds the decrement count by the increment count. At ~600 lines, it is also a calibration point for which conditioning machinery is generic: the same invariant-at-generation shape, none of the positional structure.

The generic safety metatheorem: causal witnesses, and an escrow route. The safety half of conditioning also factorizes, but the obvious formulation is *false*, and the refutation comes from the flagship safety instance itself. One would like: if updates preserve *Inv* on applicable operations and the history is honest, then *Inv* holds at every version — proved by inducting along the version's canonical enumeration. But canonicity constrains the enumeration only by *lo*, and for the bounded counter — all of whose operations commute — *lo* is *empty*: $[\text{dec}, \text{inc}]$ is a perfectly canonical enumeration of an honest version, and its intermediate state violates the escrow invariant. Only *endpoints* of canonical folds are guaranteed; commuting causal predecessors may float behind the event that needs them. The repair is to restrict the witness class, not the induction:

Definition 15.11 (causal folds; causal canonical witnesses). σ is a causal fold of E in C (*CausalFold*) when some enumeration ρ of E respecting $\xrightarrow{\text{vis}}$ folds to it: $\text{fold}(\sigma_0, \rho) = \sigma$. Prefixes of such an enumeration are exactly the vis-backward-closed subsets of E . C is causally canonical when every registered version carries a witness linearizing both orders: $\text{ver}(v) = (s, E)$ implies some enumeration ρ of E respects $\xrightarrow{\text{vis}}$ and respects lo^E and folds to s , strictly refining clause 1 of Definition 10.2. *CausalCanonical*

Definition 15.12 ($\exists$ -form honesty). For a guard A on (event, state) pairs, *HonestAppOn* $\mathcal{D}$ A C : every event satisfies A at some causal fold of its causal past,

$$\forall e \in E(C). \exists \sigma. \sigma \text{ a causal fold of } \{e' \in E(C) : e' \xrightarrow{\text{vis}} e\} \text{ in } C \wedge A(e, \sigma).$$

HonestApp $\mathcal{D}$ is the instance $A := \text{app}$: what the issuer's own materialized state witnesses. Being an $\exists$ -statement it carries its own enumeration witness, so no enumerability side condition is owed; the contrast with the $\forall$ -form *AppHonest* is Remark 15.4. *HonestApp*

Definition 15.13 (the per-datatype safety obligation). **SafetyStepOn** $\mathcal{D} \ I \ A$, over an explicit conditioning pair (I, A) : for every configuration C , event sets E, S , event e , and states σ_S, σ_P with

- (i) $E \subseteq E(C)$; (ii) E vis-closed ($a \xrightarrow{\text{vis}} b, b \in E$ imply $a \in E$); (iii) $e \in E$;
- (iv) $S \subseteq E$; (v) $e \notin S$; (vi) S vis-closed;
- (vii) S future-free for e : $x \in S$ implies $\neg e \xrightarrow{\text{vis}} x$; (viii) $\text{past}(e) \subseteq S$: $x \xrightarrow{\text{vis}} e$ implies $x \in S$;
- (ix) σ_S a causal fold of S ; (x) σ_P a causal fold of $\text{past}(e) = \{e' \in E(C) : e' \xrightarrow{\text{vis}} e\}$,

it holds that $I(\sigma_S) \wedge A(e, \sigma_P) \Rightarrow I(\text{do}(\sigma_S, e))$: applicability transfers from the causal past to any admissible prefix, fused with I -preservation so unconditionally-safe operations owe no guard. **SafetyStep** $\mathcal{D}$ is the instance $(I, A) := (\text{Inv}, \text{app})$. **SafetyStep**

Theorem 15.14 (generic safety, causal-witness form). If $\text{Inv}(\sigma_0)$, **SafetyStep** $\mathcal{D}$, and C satisfies the good-configuration invariant (Definition 10.2), is causally canonical, and satisfies **HonestApp** $\mathcal{D}$, then every registered version state satisfies **Inv**. Axioms: **propext**, **Quot.sound**.
version_inv_of_causal_canonical

Theorem 15.15 (the causal witness is free for all-comm datatypes). If all operation pairs commute and $\text{rc} \equiv \text{Either}$, then every configuration satisfying Definition 10.2 is causally canonical: a pointwise permutation argument, no reachability induction.
causalCanonical_of_all_comm_rc_either

The bounded counter discharges **SafetyStep** in three lines — extras in a prefix are other replicas' events and cannot touch the issuer's slots (**bc_safetyStep**) — and Theorem 15.10 is re-derived as a corollary, retiring the bespoke vis-maximal counting apparatus, which is thereby revealed as compensation for inducting over non-causal witnesses. A second, independent route generalizes that retired argument itself, with no causal witness and no **Inv**-preservation at all:

Definition 15.16 (measured datatypes). $\mathcal{D}$ is measured over an index type κ when it carries observations $\text{obs}_\kappa : \Sigma \rightarrow \mathbb{Z}$ and per-operation weights $\mu_\kappa : \mathcal{E} \rightarrow \mathbb{N}$ with $\text{obs}_\kappa(\sigma_0) = 0$ and the affine update law $\text{obs}_\kappa(\text{do}(\sigma, e)) = \text{obs}_\kappa(\sigma) + \mu_\kappa(e)$. Folds of measured observations compute set measures, so enumeration-independence is free. **Measured**

Theorem 15.17 (the escrow metatheorem). Let $\mathcal{D}$ be measured, k_c (consuming) and k_f (funding) indices, and A a guard, with

- (i) $\mu_{k_c}(e) \leq 1$ for every event e ;
- (ii) the guard supplies slack: $\mu_{k_c}(e) = 1$ and $A(e, \sigma)$ imply $\text{obs}_{k_c}(\sigma) + 1 \leq \text{obs}_{k_f}(\sigma)$;
- (iii) C satisfies the good-configuration invariant (Definition 10.2);
- (iv) consumption is serial: distinct events $a, b \in E(C)$ with $\mu_{k_c}(a) = \mu_{k_c}(b) = 1$ are $\xrightarrow{\text{vis}}$ -comparable (the counter obtains this from same-replica totality, per slot);
- (v) **GenHonest** $\mathcal{D} \ A \ C$ (the $\forall$ -form: measured guards are fold-order-insensitive).

Then every registered version state s satisfies $0 \leq \text{obs}_{k_c}(s) \leq \text{obs}_{k_f}(s)$. *escrow_version_inv*

Theorem 15.10 is derived a second time this way. The queue, by contrast, *refuses* **SafetyStep** — “ t is the head” is not stable under a concurrent honest dequeue of the same head, and correctly so: the double-dequeue execution satisfies any sound obligation, and the queue’s stable residue is event-level (**QHonest**), consumed by convergence rather than by a state invariant. The full analysis, including two further refuted formulations, is the pen-and-paper memo **Development/GENERIC_SAFETY_PENPAPER.md** — written, per this project’s standing lesson, before any Lean. (A natural follow-up is a *conditioned step relation* — apply guarded by **app** at the head state — under which honesty becomes a theorem of the LTS rather than a contract; deferred.)

A small fourth instance: the FWW reservation register — payload arbitration, and

what honesty cannot buy. A register claimable once (a seat, a username): sequentially the first claim wins; concurrently, the winner is the claim that is first in *Lamport* time. This is the positive complement to the metadata-free kill-test (Proposition 15.19 below): arbitration whose key is not in the state is impossible, and arbitration whose key *is* in the state is a semilattice triviality — the state is the minimum claim (ts, rep, v) under the lexicographic order (or unset), update and merge are min, and the entire eight-VC discharge is min-algebra (~ 300 lines, the smallest conditioned instance).

Theorem 15.18 (the FWW winner, per version). *In every configuration reachable in the plain LTS ($\text{Inv}(\sigma_0)$ trivial), every registered version $\text{ver}(v) = (s, E)$ satisfies*

$$(\forall e \in E. s \leq \text{claim}(e)) \wedge (s = \top \Leftrightarrow E = \emptyset) \wedge (\forall m. s = m \Rightarrow \exists e \in E. \text{claim}(e) = m) :$$

the register holds exactly the minimum-timestamp claim of its event set (a lower bound, attained), unset exactly on the empty set. No honesty hypothesis: semilattice folds are enumeration-independent.

fwv_version_min

Proposition 15.19 (metadata-free arbitration is impossible). *Let $\text{canon}(\text{init}, L)$ be the LWW specification value of a finite write history L from initial value init (the value of the timestamp-maximal write, init if L is empty). There is no function $m : \text{value}^3 \rightarrow \text{value}$ with*

$$\begin{aligned} m(\text{canon}(\text{init}, L), \text{canon}(\text{init}, L \uparrow \Delta_a), \text{canon}(\text{init}, L \uparrow \Delta_b)) \\ = \text{canon}(\text{init}, L \uparrow \Delta_a \uparrow \Delta_b) \quad \text{for all } \text{init}, L, \Delta_a, \Delta_b : \end{aligned}$$

two executions present the identical value triple $(10, 20, 30)$ with opposite stamp orders, forcing $m(10, 20, 30) = 30$ and $m(10, 20, 30) = 20$. The timestamp is forced into the state by the merge, not by replay order.

lww_merge_needs_timestamps

Since timestamps respect causality, a causally later claim never displaces an installed one; the min rule arbitrates only among *concurrent* claimants; otherwise replicas can diverge. The relevant constraint is the generation discipline: **app** for a claim is “the register is unset in my causal past”, and *deliberately no theorem consumes it* — two honest concurrent claimants both see unset, both claim, and each locally wins until a merge disabuses one. “Unset” is the minimal example of an applicability check that is *not stable under concurrent honest extension* (contrast the escrow counter’s own-slot slack, which is — precisely the boundary the safety metatheorem’s **SafetyStep** draws). A merge-based register is a reservation, confirmed only at causal stability; it is never a mutex.

The join hook is also the composition boundary. The factored interface also exposes the composition boundary: data types compose *at the per-configuration join*, and only there. Finished capstones never compose, because reachability does not project (a second-component apply is a version-duplicating stutter for the first component’s projection); per-configuration certificates do, which retroactively forces the interface of this section. The binary heterogeneous product (`Metatheory/Product.lean`) makes this a theorem pair: cross-component pairs commute definitionally, so lo^E has no mixed edges and the glued join witness is a plain concatenation of the component witnesses, with no interleaving combinatorics.

Theorem 15.20 (the product join gluing). *Let $\mathcal{D}_1 \otimes \mathcal{D}_2$ be the binary heterogeneous product (`prodSig`): state $\Sigma_1 \times \Sigma_2$, operations the sum $O_1 \oplus O_2$, with `do`, `mergeL`, `Inv`, and `app` component-wise, and `rc` per component on same-side pairs and `Either` on mixed pairs (forced: cross pairs commute, so any mixed `Fst_then_snd` would violate Flat VC 1). For any product configuration C : if `JoinLemma3At` $\mathcal{D}_1$ holds at the first core projection of C and `JoinLemma3At` $\mathcal{D}_2$ at the second, then `JoinLemma3At` $(\mathcal{D}_1 \otimes \mathcal{D}_2)$ C .* *joinLemma3At_prod*

Theorem 15.21 (the composite product metatheorem). *Let H_1, H_2 be component contracts and $H_\otimes(C) := H_1(\pi_1 C) \wedge H_2(\pi_2 C)$ the product contract over the core projections (`prodContract`). If every H_1 -configuration admits $\mathcal{D}_1$ ’s join and every H_2 -configuration admits $\mathcal{D}_2$ ’s, then, with the product’s `Inv`(σ_0) (both components’), every $H_\otimes$ -honestly reachable product configuration is RA-linearizable, per version. Axioms: `propext`, `Quot.sound`, `prod_ra_linearizable3_of_honest_reach`*

The component certificates are consumed at the *projections* of product-reachable cores, which is exactly why they must be configuration-level, not reachability-indexed. A queue $\otimes$ counter capstone with zero bespoke proof (`MRDT_Instances/ProductDemo/`) demonstrates consumability; the safety and $\approx$ -quotient kits, the boundary findings, and what remains open are collected in Open Question 19.

16 Virtual LCAs: merging without a common-ancestor version

The transition system of §3 gates Merge on the existence of an LCA: a version v_ℓ that reaches both heads and dominates every common ancestor. In a *criss-cross* configuration the common ancestors of two heads have two or more maximal elements, no version satisfies the predicate, and Merge is disabled. This is not an exotic corner. The shape’s routine genesis is two replicas each merging the *same* pair of diverged heads: both create a version with the same event set, neither reaches the other, and every later pair of descendants has both rivals as maximal common ancestors (Figure 6). The JS runtime built on this model makes the gate explicit, and its randomized testing shows how routine the shape is under honest peer-to-peer syncing: 592 gated sync attempts in 220 randomized trials. The gap is exactly where git reaches for its *recursive* merge strategy: merge the maximal common ancestors into a virtual base, then merge the heads against that. This section develops that extension inside the metatheory and reports what it costs: nothing, for join-lemma datatypes.

The rule. Write $\text{MCA}(v_1, v_2)$ for the maximal common ancestors of two versions (maximal in the reachability order, which is a partial order because a version’s identifier is its allocation rank). $\text{VirtualLCA}(v_1, v_2)$:

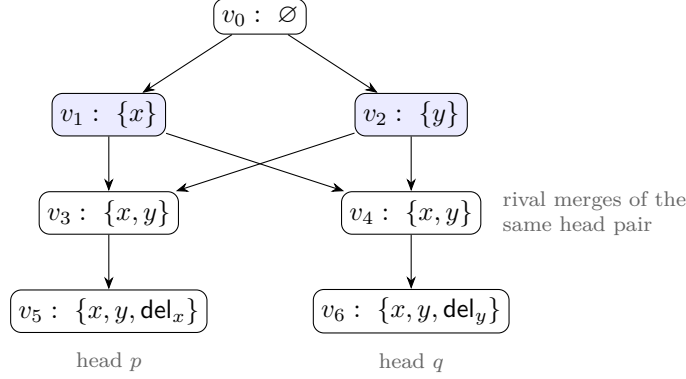


Figure 6: The criss-cross store used for the machine pins (the store of `VirtualLCA_Spot.lean`, drawn top-down; edges parent to child). Replicas p and q concurrently add x and y ; each then merges the same diverged pair (v_1, v_2) , creating the rivals v_3, v_4 with equal event sets; each deletes one element. The common ancestors of the heads v_5, v_6 are $\{v_0, v_1, v_2\}$ with *two* maximal elements v_1, v_2 : no version is an LCA, and the gated Merge cannot fire. Note $E(v_1) \cup E(v_2) = \{x, y\} = E(v_5) \cap E(v_6)$: the antichain’s union is exactly the meet, which is Proposition 16.4.

- $M := \text{MCA}(v_1, v_2)$. It is nonempty (the pinned root is a common ancestor), and when the heads are incomparable every member is a strict ancestor of both.
- If $M = \{m\}$: return m . This is the existing rule, and Lemma 4.2 applies unchanged.
- Otherwise sort M ascending by rank (deterministic: version ids are allocation ranks) as $m_1 < \dots < m_k$ and fold: start with m_1 ; at each step, merge the accumulator with the next member m_i through the recursively resolved LCA of the sub-pair, materializing a *scratch node* whose state is that ternary merge and whose event set is the union so far. Return the final accumulator.

The head merge then runs the ordinary ternary merge with the returned state in the LCA slot. The scratch nodes exist so the nested MCA queries are well posed; nothing references them afterwards, so they are never ancestors of real heads and cannot perturb later queries. Termination is unconditional: measuring a call on (a, b) by the number of allocated versions in the pair’s joint ancestry, every version in a sub-pair’s support is a common ancestor of the original heads while v_1 itself is not, so the measure strictly drops at each nesting level, and within a level the fold index drops. In the mechanization the extension lands as a conservative *new* step relation **Step3V**, with a base case embedding every old step and **mergeVirtual** the one new case; widening **Step3** in place was tried and rejected, because it breaks every exhaustive case analysis over the old relation in instance files. Only the final virtual node is allocated; the store invariant extends across the new step. The next three definitions pin the rule formally.

Definition 16.1 (common ancestors and MCAs, set-support form). *The recursion pairs a scratch node, identified with its finite real support S , against a version w ; $S = \{v_1\}$ recovers the head pair. Over the parent relation,*

$$\text{CommonAnc}(S, w) = \{x : (\exists u \in S. \text{Reaches}(x, u)) \wedge \text{Reaches}(x, w)\},$$

and m is a maximal common ancestor (lsMCA) of (S, w) when $m \in \text{CommonAnc}(S, w)$ and every $x \in \text{CommonAnc}(S, w)$ with $\text{Reaches}(m, x)$ equals m . The antichain is materialized as a

finite set (*mcaFinset*; ranks are bounded by w , so the bound is intrinsic), and LCAs are exactly the singleton case (*isMCA_singleton_of_isLCA* and its converse). *CommonAnc, IsMCA*

Definition 16.2 (the virtual-LCA state). Write σ_v for the state registered at v , defaulting to σ_0 (*stateD*; the canonicity theorems carry the allocation hypotheses that make the default unreachable). Sort $\text{MCA}(S, w)$ ascending by rank as $m_1 < \dots < m_k$; then the recursion of the rule above is (*vlcaAux*, *vfoldAux*):

$$\begin{aligned} \text{vlca}(S, w) &= \begin{cases} \sigma_0 & k = 0 \\ \text{vfold}(\{m_1\}, \sigma_{m_1}, [m_2, \dots, m_k]) & k \geq 1 \end{cases} \\ \text{vfold}(A, \text{acc}, []) &= \text{acc} \\ \text{vfold}(A, \text{acc}, m :: ms) &= \text{vfold}(A \cup \{m\}, \text{mergeL}(\text{vlca}(A, m), \text{acc}, \sigma_m), ms), \end{aligned}$$

and $\text{VirtualLCA}(v_1, v_2) := \text{vlca}(\{v_1\}, v_2)$. The empty-antichain base is not a junk case: under the store invariant its event set, the empty union, is exactly the then-empty meet. Noncomputable but total, by the rank measure of the termination argument above. *virtualLCASState*

Definition 16.3 (the widened transition system). *Step3V* embeds every *Step3* rule (Definition 3.2) through a base constructor and adds one rule, $\text{MERGEVIRTUAL}(r_1, r_2)$:

$$\frac{\begin{array}{l} \text{head}(r_1) = v_1 \quad \text{head}(r_2) = v_2 \quad \text{ver}(v_i) = (\sigma_i, E_i) \\ \text{ver}(v_m) \text{ unallocated} \quad v_1 < v_m \quad v_2 < v_m \end{array}}{\begin{array}{l} \text{ver}(v_m) := (\text{mergeL}(\text{VirtualLCA}(v_1, v_2), \sigma_1, \sigma_2), E_1 \cup E_2), \\ \text{parents}(v_m) := [v_1, v_2], \quad \text{head}(r_1) := v_m \end{array}}$$

with visibility unchanged: this is *MERGE* of Definition 3.2 with the *isLCA* gate and its $\text{ver}(v_\ell)$ read replaced by the recursive antichain merge in the LCA slot; with a singleton antichain the virtual state is the registered LCA state, so the rule strictly generalizes *MERGE*. The widened LTS is *labeledTS3V*. *Step3V*

The covering proposition. The Merge rule's semantic requirement on the LCA slot is Lemma 4.2: its event set must be the head intersection. The virtual construction meets it exactly, on the strength of invariants the store already carries.

Proposition 16.4 (covering). Let $\text{ver}, \text{parents}$ be a raw store with (i) the store invariant of Definition 4.1, (ii) rank-decreasing parents ($p \in \text{parents}(v)$ implies $p < v$), (iii) every $u \in S$ allocated, and (iv) w allocated. Then, pointwise in the event e ,

$$(\exists m. \text{IsMCA}(S, w, m) \wedge e \in E(m)) \iff (\exists u \in S. e \in E(u)) \wedge e \in E(w);$$

by extensionality,

$$\bigcup_{m \in \text{MCA}(S, w)} E(m) = \left(\bigcup_{u \in S} E(u) \right) \cap E(w).$$

Neither finiteness nor nonemptiness of S is assumed, and the degenerate cases come for free: an empty MCA set forces an empty intersection (the $\supseteq$ direction constructs an MCA from a shared event's origin), and a singleton MCA set recovers Lemma 4.2. *mca_events_cover*

With $S = \{v_1\}$, $w = v_2$ this covers the head pair; with S the support of a scratch node it covers every sub-pair of the recursion; with a singleton MCA it degenerates to Lemma 4.2. The subset direction is event-set monotonicity along reachability, twice. The superset direction is where the store invariant earns its keep: an event in both sides has, by the *origin* invariant of §4.1 (each event appears first at a unique generator version), a single birth version that reaches every version whose event set contains it; that generator is a common ancestor, every common ancestor reaches a maximal one, and monotonicity carries the event up into it. Since the fold unions event sets, the returned virtual node has event set $\bigcup_i E(m_i) = E(v_1) \cap E(v_2)$: exactly what the Merge rule requires. A natural expectation (the GC analysis of §17) is that the union only approximates the intersection from below; under the carried invariants the approximation is *exact*, even though each antichain member individually sits strictly below the meet.

One delimitation marks the model boundary the proposition rests on. The origin invariant is load-bearing: in a model that can apply the *same* event at two incomparable versions (op-based redelivery, with no store-wide timestamp freshness), the event has two incomparable birth sites, neither a common ancestor, and the union strictly undershoots the intersection. Countermodel: apply e independently at the root on both branches; then $E(v_1) \cap E(v_2) = \{e\}$ while the sole MCA is the root with empty event set. The transition system excludes this by store-wide timestamp freshness (an op is born at exactly one version), so the covering invariant is not a new obligation: it is `StoreInv.origin` plus monotonicity, already carried and maintained.

Canonicity: one fold induction, and the VC surface does not move. The adequacy induction of §10 consumed two facts about the LCA slot: its event set is the intersection (now Proposition 16.4) and its registered state is canonical for that event set. The virtual construction re-supplies the second from the one per-datatype hypothesis the framework already centers on, the per-configuration Join Lemma:

Claim 16.5 (virtual join). *Let C satisfy (i) the store invariant on its ranked store (Definition 4.1), (ii) the good-configuration invariant (Definition 10.2; in particular every allocated version's state is canonical for its event set), and (iii) the per-configuration Join Lemma JoinLemma3At at its core (Definition 5.3). Then, for any registered pair $\text{ver}(v_1) = (s_1, E_1)$, $\text{ver}(v_2) = (s_2, E_2)$,*

$$\text{Can}_C(E_1 \cap E_2, \text{VirtualLCA}(v_1, v_2)),$$

and along the fold every scratch node's state is canonical for its union event set.
virtualLCASState_canonical

The proof is an induction over the ascending-rank fold: the antichain members are allocated, hence canonical by hypothesis; at each step the sub-pair's inner LCA slot is canonical by the inner recursion plus Proposition 16.4, and the Join Lemma yields canonicity at the union. Backward closure is preserved by union and intersection, so all intermediate sets stay in the hypothesis class; and canonicity is a property of a (configuration, event set, state) triple, not of allocated versions, so scratch states need no allocation to carry it. The same induction instantiates at the witness-classed join (***virtualLCASState_canonicalF***), and the merge case of the honest-reachability metatheorem gains the sibling case:

Theorem 16.6 (the widened metatheorem). *Let HonestReachV be honest reachability (Definition 15.1) with Step3 replaced by Step3V (Definition 16.3). If every H -configuration admits*

the Join at its core, then, with $\text{Inv}(\sigma_0)$, every H -honestly reachable configuration of the widened LTS is RA-linearizable, per version: the same join obligation. The induction gains one case (*goodConfig3_mergeVirtual_at*), whose extra store-invariant hypothesis reachability itself supplies (*storeInv_of_honest_reachV*). *ra_linearizable3_of_honest_reachV*

The per-datatype proof interface does *not* change: `Framework/VC_Set.lean` is untouched. For any datatype that discharges its join at every configuration, the criss-cross gate lifts with **no new per-datatype verification condition**; the per-datatype VC surface does not move.

Two statement-hygiene facts, both machine-witnessed. First, determinism: the fold order is fixed (ascending rank), and for join-lemma datatypes order-insensitivity of the *returned* state is a theorem, since canonical states of a fixed event set are unique. Git’s own recursive strategy folds in commit-date order and is known to be order-sensitive on conflicting merges; rank fold plus canonicity is what removes that defect here. Second, the insensitivity claim must be scoped to the result: on a directed three-member antichain the *intermediate* scratch states of the ascending and descending folds genuinely differ (each is canonical for its own, different, union), and only the final states coincide. Any mirror lemma stating canonicity of intermediates across fold orders is false.

The single-MCA shortcut, machine-refuted. The tempting implementation shortcut (pick *some* maximal common ancestor and use it as the LCA; no recursion, no scratch nodes) is not a simplification but a soundness bug, and the store of Figure 6 pins it. Resolving the head pair (v_5, v_6) through the virtual base folds v_1 and v_2 over the root into the state with both adds visible; the head merge then sees each delete on its own side against a base that *contains* the deleted element, and both deletions survive: the merged state is empty, the meet’s fold. Picking the single MCA v_1 instead puts a base without y in the LCA slot, so y on v_6 ’s side reads as a fresh add and the deleted y *resurrects*; picking v_2 dually resurrects x .

Proposition 16.7 (every single-MCA pick resurrects a deleted element). *On the store of Figure 6 (`Metatheory/VirtualLCA_Spot.lean`, a two-element OR-pair datatype; query _{y} is the membership read of y , σ_v the registered states, and the pair resolved is the heads (v_5, v_6)):*

$$\text{query}_y(\text{mergeL}(\sigma_{v_1}, \sigma_{v_5}, \sigma_{v_6})) = \text{true} \quad \wedge \quad \text{query}_y(\text{mergeL}(\text{VirtualLCA}(v_5, v_6), \sigma_{v_5}, \sigma_{v_6})) = \text{false},$$

and dually the pick v_2 resurrects x (*t1f_pick_v_resurrects_x*); neither pick’s merged state equals the virtual resolution’s (*t1f_picks_ne*). The harness twin is the T1F fail companion of *virtual_lca_check.py*. *t1f_pick_u_resurrects_y*

The recursion is necessary, and a fixed depth bound is unsound in practice. In a dense randomized probe (200 trials, 6 replicas, sync-heavy), criss-crosses arose in 176 of 200 trials with resolution nesting up to depth 7, because rivalry propagates upward (rival joins of rival joins); a depth-limited implementation would reject valid merges beyond its bound.

The quotient layers, and the rehoming stress test. The claim above serves the raw layer. The framework’s quotient layers (the $\approx$ -indexed contracts of §14) each thread their own merge case, and their bill was probed on the hardest instance available: the rehoming RGA, the one production datatype whose proof route runs through an equivalence quotient with a reachability-anchored honesty stack (its only role here; it is the generality stress test, nothing more). Two

findings. First, the dividing line at antichain unions is exactly the per-event versus prefix-state line: honesty components that speak about an event’s *own* causal past (born accuracy, dependency-prefix correctness) restrict to every scratch union for free, and the probe confirms them at every union of 590 randomized trials; components that speak about *applicability along an enumeration* (`noopFeasible`, applicable delivery) are *false* at antichain unions, in three named shapes (an insert whose anchor a concurrent kill has already removed fired 390 times across the probes; stale delete and insert paths fired 68 and 80 times), while every one of those states remained canonical for its union. So union-level obligations must be generation-discipline shaped, never applicability shaped. Second, the bill collapsed on execution: exactly *one* fold induction was owed beyond the raw layer (at the H layer, whose merge case consumes the registered slot’s layer canonicity); the Eq-quotient layers contain no step destructs and re-thread mechanically (`rga_ra_linearizable3_eq_V`, and the flat production catalogue over `Step3V` via `FlatGeneric_Bridge_V`); and the NF layer is precisely the refuted `noopFeasible` route with no production consumer, so its mirror was *deliberately not built*. The refuted predicate marks a dead proof route, not a defect of the construction.

The queue, as a corollary. The mergeable queue of §15 illustrates reuse: its join is already per-configuration (`q_join_at`), which is exactly the hook the virtual fold induction consumes, so lifting the criss-cross gate costs the queue nothing beyond restating reachability over the widened LTS. The capstone `queue_ra_linearizable3_V` (RA-linearizability at every honestly reachable configuration with virtual merges enabled) is ten lines, all of them plumbing: the framework extension multiplies across instances at the join boundary.

17 Commit GC: the keep set and the GC-safety theorem

The version store only ever grows, and merges arbitrarily far in the future reach back into it for their LCA state, so a runtime cannot reclaim “everything below the heads.” The GC question is which store entries may be dropped without changing any future behavior; the answer is a keep set computable from the current heads, and a forward simulation showing pruning outside it is invisible.

The keep set. Fix a configuration C_0 (the prune point). A version is a *head* (`IsHead`) when some replica’s head map points to it; write H for the current heads.

Definition 17.1 (the keep set). *Over allocated versions (heads and v allocated, with the displayed event sets),*

$$\text{Keep}(C_0) = \{ v : \exists h_1, h_2 \in H \ (h_1 = h_2 \text{ allowed}), \ E(h_1) \cap E(h_2) \subseteq E(v) \},$$

$$\text{Keep}'(C_0) = \text{Keep}(C_0) \cup \{0\},$$

and $\text{Dropped}(C_0) = \{ v \text{ allocated} : v \notin \text{Keep}'(C_0) \}$ is what pruning removes. **keepSet,**
keepSet', droppedSet

This is the *upward event-set closure of the pairwise head intersections*, with the root 0 pinned separately (the model fixes $\text{ver } 0 = (\sigma_0, \emptyset)$ forever, and fresh replicas are created there). Taking $h_1 = h_2$ shows every head is kept, and with it everything event-set-above any single head.

Pruning drops the store *payload* only: entries outside the keep set become unallocated, while the replica cores and the DAG skeleton (parents, a map of ids, not states) are untouched. Retaining the skeleton is forced, not a convenience: restricting the parent relation to kept versions changes the LCA predicate itself, since removing common ancestors makes the domination clause easier to satisfy, so new LCA triples can appear whose event sets violate Lemma 4.2, and the pruned structure would fail its own invariant. The reclaimable memory is the states and event sets, which is where the memory is; the skeleton stays.

The theorem. Safety is a forward simulation, built from three lemmas, the first two packaged as a run invariant:

Definition 17.2 (the GC run invariant). $\text{HeadDom}(C_0, e)$: some allocated prune-time head's event set is contained in e . $\text{Virgin}(C_0, v)$: every prune-time-allocated ancestor of v (along the current parent relation) is the root. $\text{GCInv}(C_0, C)$:

- (i) *permanence*: allocation is permanent and entries immutable, so v allocated in C_0 implies $\text{ver}_C(v) = \text{ver}_{C_0}(v)$;
- (ii) *monotonicity (disjunctive)*: every current head v with $\text{ver}_C(v) = (s, e)$ satisfies $\text{HeadDom}(C_0, e) \vee \text{Virgin}(C_0, v)$;
- (iii) the store invariant of Definition 4.1 at C .

GCInv, HeadDom, Virgin

Monotonicity is clause (ii): along any run from C_0 , every current head either event-set-dominates some prune-time head, or its only prune-time ancestor is the pinned root. The disjunction is forced by replica creation: a fresh replica's head is the root, and its lineage grows from empty until it first merges with a dominating lineage, at which point it becomes dominating itself. *Intersection containment*: if a future merge fires with heads v_1, v_2 and allocated LCA v_ℓ , then v_ℓ was not dropped; when both sides dominate prune-time heads h_1, h_2 , Lemma 4.2 gives $E_0(h_1) \cap E_0(h_2) \subseteq E(v_1) \cap E(v_2) = E(v_\ell)$, which is membership in the keep set, and when either side is virgin the LCA can only be the pinned root (`lca_not_dropped`). *Prune commutation*: every step of the unpruned run is matched, with the same label, on the pruned store; apply reads only the issuing head and a fresh slot (never dropped: dropped slots were allocated at prune time and allocation is permanent), and merge reads the two heads and the LCA, which the previous lemma keeps.

Theorem 17.3 (GC safety). *Let C_0 satisfy the store invariant (supplied by plain reachability, Theorem 4.3), and let $C_0 \xrightarrow{\ell_1 \dots \ell_n} C$ be any labelled Step3 run. Then the pruned prune point admits the same run against the correspondingly pruned endpoint,*

$$C_0 \setminus \text{Dropped}(C_0) \xrightarrow{\ell_1 \dots \ell_n} C \setminus \text{Dropped}(C_0) \quad (\text{pruneKeep}, \text{dropVer}),$$

with the same label sequence (in particular every query, whose value rides on the label), and every replica read at every point is unchanged: $N_{C \setminus \text{Dropped}(C_0)} = N_C$. Prune-definedness is convention (iv) of §3.

gc_safety

One scope note, recorded because it is a genuine asymmetry: the converse simulation is deliberately not mechanized. A pruned store can exhibit steps the unpruned one cannot (a freed slot can be re-allocated; store-wide timestamp freshness quantifies over fewer entries), and the asymmetry is unobservable only via a reachability invariant (every allocated version sits below some current head) that is not part of the configuration structure. The direction GC-safety needs (nothing is lost) is the mechanized one.

The head-sync countermodel. Head-sync is the discipline that future merge arguments are current heads; the Merge rule enforces it syntactically. The keep set is calibrated to exactly that discipline, and the calibration is tight: a merge from an old interior version genuinely needs a version outside the keep set. The countermodel, worked concretely (replicas A, B ; events e_1, e_2, e_3 on A , e_4 on B):

version	created by	events	parents	kept?
0	root	$\emptyset$	$[]$	pinned
1	A applies e_1 at 0	$\{e_1\}$	$[0]$	dropped
2	A applies e_2 at 1	$\{e_1, e_2\}$	$[1]$	kept
3	B merges A (LCA 0)	$\{e_1, e_2\}$	$[0, 2]$	kept
4	A applies e_3 at 2 (head)	$\{e_1, e_2, e_3\}$	$[2]$	kept
5	B applies e_4 at 3 (head)	$\{e_1, e_2, e_4\}$	$[3]$	kept

The pairwise head intersection is $E(4) \cap E(5) = \{e_1, e_2\}$; version 1, with $E(1) = \{e_1\}$, contains no pairwise intersection and is dropped, strictly (the store genuinely returns nothing at 1 after pruning). The future head merge is safe: the LCA of (4, 5) is version 2, kept. But version 1 satisfies the LCA predicate for the *interior* pair (1, 5), so any widened Merge that accepted a non-head argument (a rewind replica, a checkpoint restore, a replica missing from the head map) would demand the dropped entry. Every out-of-band way of resurrecting an old version breaks the keep set; head-sync is a hypothesis of the metatheorem, not pessimism. The countermodel is machine-checked:

Proposition 17.4 (head-sync is load-bearing). *In the store C_{ex} of the table above (*GC_Safety.lean*, over the framework-resident conditioned grow-only set),*

$$\text{lsLCA}(1, 5, 1) \wedge \neg \text{lsHead}(C_{\text{ex}}, 1) \wedge 1 \in \text{Dropped}(C_{\text{ex}}) \wedge \text{ver}_{\text{pruned}}(1) = \text{none} :$$

the dropped interior version is the LCA of the interior pair (1, 5), so a non-head-sync merge demands a pruned entry.

gc_needs_head_sync

The revision under virtual merges. §16 widens what a merge reads: where no exact LCA exists, the resolution reads the maximal common ancestors, whose event sets sit strictly *below* the pairwise meet, while the keep set above retains versions *above* some pairwise meet. Under the widened rule the original keep set prunes away exactly what the recursion needs. The revision:

Definition 17.5 (the MCA closure, the revised keep set, and the widened invariant). *mcasClosure(C) is the least set of versions containing every allocated head and closed under pairwise MCAs of its members (inductively: rules HEAD and MCA, the latter over $\text{lsMCA}(\{a\}, b, m)$*

for members a, b ; finite, since ranks only decrease). The revised keep set is the upward event-set closure of its members, plus the pinned root:

$$\text{Keep}_V(C_0) = \{ v \text{ allocated} : \exists w \in \text{mcasClosure}(C_0) \text{ allocated, } E(w) \subseteq E(v) \} \cup \{0\}.$$

The run invariant GCInv_V extends Definition 17.2 with: the prune-time store invariant; parent-list immutability at prune-time versions; the head gate (a current head allocated at prune time is a prune-time head or the root); and the gateway invariant: for every v allocated after the prune point and every prune-time-allocated w with $\text{Reaches}(w, v)$, either $w = 0$ or some prune-time head h has $\text{Reaches}(w, h)$ and $\text{Reaches}(h, v)$. *mcasClosure, keepSetV, GCInvV*

Keep_V contains the old keep set, and covers every recursion read: sub-pair MCAs are contained in real-pair MCAs, iterated. The one-layer seed (the pairwise MCAs of the current heads, without the fixpoint) is one closure layer short. On a machine-checked directed nested criss-cross, a depth-2 resolution reads the MCAs of the MCAs, pruning by the one-layer rule kills them (the sync raises a pruned-store error), and pruning by the closure rule keeps them while still pruning a genuinely dead interior commit, with the post-GC sync equal to the no-GC control. Future-proofing (the analogue of intersection containment) needs one new reachability fact, the *gateway* lemma: after the prune point, every ancestry path from a prune-time version to a post-prune version passes through a prune-time head or the pinned root, so a prune-time version that a future virtual resolution reads is an MCA of prune-time heads (layer one of the closure), and deeper reads stay in the closure.

Theorem 17.6 (GC safety, widened LTS). *Let C_0 satisfy the store invariant, and let $C_0 \xrightarrow{\ell_1 \dots \ell_n} C$ be any labelled Step3V run (gated and virtual merges, Definition 16.3). Then, pruning to $\text{Keep}_V(C_0)$,*

$$C_0 \setminus \text{Dropped}_V(C_0) \xrightarrow{\ell_1 \dots \ell_n} C \setminus \text{Dropped}_V(C_0)$$

as a Step3V run with the same label sequence, and every replica read at every point is unchanged. The statement matches Theorem 17.3; the gateway changes the proof, not the theorem interface.
gc_safetyV

Bounded state: dropping the skeleton by a clock swap. The theorem above reclaims the payload but keeps the parent map, so the pruned store still costs $O(\text{commits ever minted})$: it retains history's shape, not its states. The skeleton was forced only for the parent-relation representation, where restricting parents mints new LCA triples that falsify `lca_events` (the argument of the keep-set paragraph). A change of representation removes it. The commit graph is a delta-compression of event sets, so a version's ancestry role is already carried by its event set used as a *clock*: ancestry is $\subseteq$, and the LCA of a head pair is the kept version at the *clock meet* $E(v_1) \cap E(v_2)$, which is exactly what `lca_events` says the structural LCA carries. The bounded store keeps only the kept payloads and sets `parents` $\equiv []$; with no edges, structural reachability collapses to equality and `lca_events` holds trivially, so there is no history to store and the state is $O(|\text{Keep}'|)$ (`GC_BoundedState.lean`).

Definition 17.7 (the bounded store and the clock lookup). `clockPrune(C)` *prunes the payload to $\text{Keep}'(C)$ and sets `parents` $\equiv []$: its `ver` support is contained in $\text{Keep}'(C)$ (`clockPrune_support`) and its skeleton is the constant empty function (`clockPrune_no_skeleton`). `ClockLCA(cc, v1, v2, u, su)`: the heads are allocated with event*

sets e_1, e_2 and $\text{ver}_{cc}(u) = (s_u, e_1 \cap e_2)$; the lookup reads event sets only, never parents. $\text{ClockCoherent}(cc)$, the lineage guard: any two allocated versions with the same event-set clock carry the same state (the payload is a function of the clock). *clockPrune, ClockLCA, ClockCoherent*

Theorem 17.8 (the clock lookup is complete; under the guard, sound). *Completeness (guard-free): if C satisfies the store invariant and a head pair v_1, v_2 (both allocated, at heads r_1, r_2) has a structural LCA v_T with $\text{ver}(v_T) = (s_T, e_T)$, then $\text{ClockLCA}(\text{clockPrune}(C), v_1, v_2, v_T, s_T)$: the swap never disables a merge the skeleton enabled (the LCA is kept, *lca_not_dropped*). Soundness (guarded): if $\text{ClockCoherent}(cc)$ and both $\text{ClockLCA}(cc, v_1, v_2, u, s_u)$ and $\text{ClockLCA}(cc, v_1, v_2, u', s_{u'})$, then $s_u = s_{u'}$. Combined: under the guard, every clock LCA of a head pair in the bounded store carries exactly the structural LCA's state (*clockLCA_recovers*). *clockLCA_complete, clockLCA_sound**

Theorem 17.9 (bounded-state GC safety). *Let C_0 satisfy the store invariant and $C_0 \xrightarrow{\ell_1 \dots \ell_n} C$ be any Step3 run. On the bounded LTS Step3C (every rule of Definition 3.2 with parents frozen at $[]$, and MERGE reading its LCA state by ClockLCA instead of lsLCA plus $\text{ver}(v_\ell)$):*

- (i) $\text{clockPrune}(C_0) \xrightarrow{\ell_1 \dots \ell_n} C \setminus_b \text{Dropped}(C_0)$, the whole trace, label for label (the flat drop $\setminus_b$ also empties the skeleton);
 - (ii) every replica read at every point is unchanged;
 - (iii) the bounded store's payload support is contained in $\text{Keep}'(C_0)$; and
 - (iv) its parent map is constantly empty,
- so the state costs $O(|\text{Keep}'|)$, not $O(\text{commits ever})$. *gc_safety_bounded*

Root-free retained-DAG compression. The clock store above is not the only bounded representation. The production commit GC now uses the more direct retained graph: its edge relation connects two retained versions exactly when the first reached the second before collection. Thus the carrier contains no dropped identifier (in particular no distinguished root unless it is retained), while one summarized edge replaces an arbitrarily long dropped path. `compressedReaches_iff` proves exact reachability between retained versions; `compressed_isLCARetained_iff` proves exact common-ancestor/LCA answers in the retained induced order; and `gc_safety_compressed` packages these graph facts with the label/read preservation of `gc_safety`. The executable DAG filters boundary parent references after deleting payloads, which is the sparse upward-closed instance of this specification (`GC_CompressedDAG.lean`).

Distributed evidence protocol. The single global configuration does not model an asynchronous implementation. The distributed semantics in `Distributed_GC.lean` does: each replica has its own head, commit holding, closed roster, and monotone frontier-evidence map. A fetch unions a remote advertisement into the receiver's commits and evidence. Fetch does not move the local head; a separate head-sync step installs an advertised version that the receiver holds. Local collection refuses when any non-self roster member lacks evidence. Membership removal is restricted to a member with no authored commit or evidence, and a parent-free epoch

base is ingested as a complete snapshot. Finally, `evidence_ancestor_of_later_head` records the own-head discipline that keeps advertised evidence below every later author head.

The refinement target is a world of independent no-GC stores, not one global store at every intermediate point. A local collection can make replica holdings unequal, so exact projection to one global carrier is false between asynchronous steps. At a coordinated cut, equal certified keep sets recover that projection (`coordinated_collect_projects_global`).

Theorem 17.10 (asynchronous distributed GC refines no-GC). *For every finite execution of fetch, head-sync, and locally certified collection steps from a compact world C_0 related pointwise to a no-GC world F_0 , there is a no-GC execution from F_0 with the same fetch and head-sync actions. Collection steps erase to stuttering steps, the final worlds remain pointwise related, and every replica reads the same head state in both worlds. `distributed_execution_refines_noGC`, `world_read_preserved`*

The operational MRDT layer supplies the second refinement. Its `DistributedConfig` pairs the semantic configuration with one physical store per replica. A visible semantic step is enabled only when the issuing store holds every version that the step reads. Fetch and GC are silent. Erasing these silent labels yields an ordinary conditioned execution (`distributedConfig_refines_Step3`); a query is unavailable when its physical store lacks the semantic head (`query_unavailable_without_head`). The unified certificate lifts the result to generation honesty, RA-linearizability, the datatype safety invariant, and client observations (`UnifiedVerifiedMRDT.distributed`).

Fetch also carries the liveness acknowledgement used by history pruning. A receipt is accepted only for a peer’s advertised current head that the observer holds. Recording it changes no semantic state (`acknowledge_erases`); any interleaving of receipts and core protocol steps still refines no-GC (`acknowledged_execution_refines_noGC`). Thus a quiescent peer can acknowledge an epoch during an ordinary fetch without minting a fake RDT operation (`pruneEvidence_of_receipts`). An empty receipt book cannot manufacture evidence (`no_pruneEvidence_without_authored_or_receipt`).

The swap has exactly one hazard, and it is the lineage collision that §18 isolates. Resolving the LCA by clock alone is sound while the payload is a function of the event set, and unsound once datatype compaction breaks that: a same-clock version from an unrelated lineage, used as a merge’s L , resurrects a dropped instance. The structural `IsLCA` sidesteps it by resolving within shared ancestry; a bare clock lookup cannot, so it is guarded by `ClockCoherent` (Definition 17.7), which is precisely the R_S -compatibility of §18; soundness under the guard is Theorem 17.8. The boundary is machine-checked both ways: a PASS SPOT where a future head-pair merge finds its LCA at the clock meet and reads correctly, and a FAIL SPOT:

Proposition 17.11 (the lineage collision). *On the compacted store C_{coll} (`GC_BoundedState.lean`, two lineages compacted to the same clock),*

$$\begin{aligned} &\text{ClockLCA}(C_{\text{coll}}, 3, 4, 1, \{10\}) \wedge \text{ClockLCA}(C_{\text{coll}}, 3, 4, 2, \emptyset) \\ &\wedge \{10\} \neq \emptyset \wedge \neg \text{ClockCoherent}(C_{\text{coll}}) : \end{aligned}$$

the head pair (3, 4) ’s clock lookup returns two same-clock candidates with divergent states, so clock identification alone resurrects the dropped instance (picking version 1) or loses it (picking 2), and the guard is provably false on this store; conversely, had the guard held, Theorem 17.8 would have forced the two lookups equal (`spot_guard_would_fix`). `spot_lineage_resurrection`

One case is deliberately not covered: a criss-cross antichain of MCAs has no single clock meet, so the bounded form of a *virtual* merge needs the retained lineage among the MCA closure, and is left owed.

One frontier, three consumers. The meet of the current heads, $\bigcap_{h \in H} E(h)$, is the *stable cut*: the events every replica has already incorporated, below which no future merge can ever again place a concurrent event. Three parts of this document consume that one frontier. The garbage collector reads it from *above*: the keep set retains the upward closure of the pairwise meets, and what pruning reclaims is exactly the versions no future LCA lookup can name. The stability VC of §18 reads it from *below*: facts certified at the cut stay true under all future merges, which is what licenses compaction. And the storage layer of Part II (§) consumes it as the epoch boundary for coordinate re-coding, where the in-flight-anchor argument needs visibility of all current heads. A runtime that tracks the frontier once serves all three consumers from the same bookkeeping, at pairwise rather than global granularity where GC needs it.

18 The stability VC: verified GC callbacks

Commit GC (§17) reclaims *versions*; it never touches the states themselves. But states also accumulate redundancy: an OR-set holding two add instances of the same element needs only one once both are known everywhere, and the embedded-chain RGA’s coordinates carry dead history that a re-coding epoch could shed (§). The runtime therefore wants to call back *into the datatype* so the concrete state can compact. The question is what the datatype must prove for such a **compact** to be sound, and what exactly the runtime must promise it. Countermodels determine both sides of the interface.

The naive gate is unsound: the discriminating remove. The obvious runtime promise (“the cut S is contained in every current head”) does not suffice, by a near miss that shapes everything after it. Adds of element a at stamps $t_1 < t_2$, both below every head. Replica A compacts, dropping the older instance t_1 . But a remove, minted long ago by a replica that had observed *only* t_2 (the remove is concurrent with the add t_1), is still at or below some head and arrives later. In the full execution the remove kills only t_2 and the element survives through t_1 ; in the compacted execution t_1 is gone and the element is absent. Reads diverge (Figure 7). The countermodel fires mechanically in the validation harness under the naive gate, and once (seed 2026, trial 218) it was found by the randomized twin runs rather than injected. It is also machine-checked in Lean, on the concrete merge states:

Proposition 18.1 (the discriminating remove). *Let l_V hold both adds of a live (the shared LCA payload of Figure 7), a_{naive} the naively compacted head (older instance t_1 dropped), and b_{rem} the branch where the in-flight remove killed only t_2 . Then*

$$a \in \text{read}(\text{mergeL}(l_V, l_V, b_{\text{rem}})) \quad \wedge \quad a \notin \text{read}(\text{mergeL}(l_V, a_{\text{naive}}, b_{\text{rem}})) :$$

the control and the naively compacted merge diverge on a
(MRDT_Instances/ORSet/ORSet_Stability.lean). countermodel_diverges

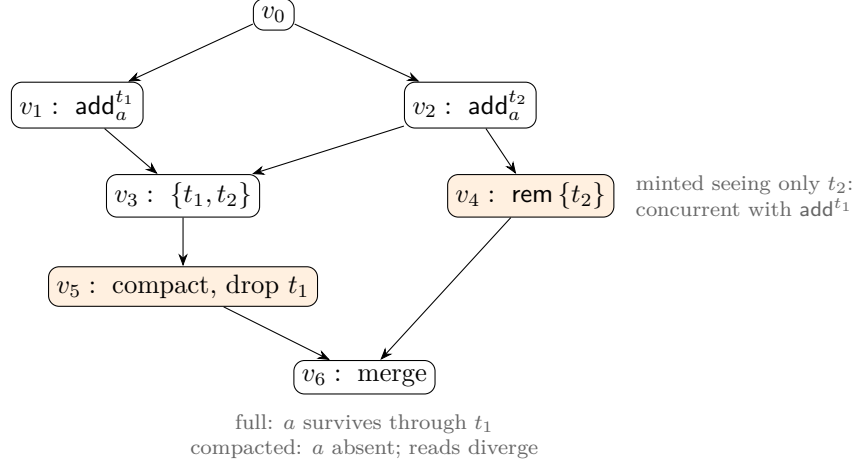


Figure 7: The discriminating remove (top-down; edges parent to child). Both adds of a sit below every head of the compacting replica, yet an in-flight remove that observed only t_2 can still arrive. Dropping the older instance t_1 under the naive “below every head” gate changes the read: the remove kills t_2 alone, and only the full state still holds a . The repair is `SettledAt`: compaction must wait until every event concurrent with the cut is already absorbed.

The interface: `SettledAt`. The repair is a version-level condition.

Definition 18.2 (`SettledAt`). Events e, a are concurrent in C (*ConcOp*) when $e \neq a$ and neither is $\xrightarrow{\text{vis}}$ the other. A cut S is settled against an event set E (*SettledAtOn*) when

- (i) $S \subseteq E$;
- (ii) S is causally downward closed ($a \xrightarrow{\text{vis}} b, b \in S$ imply $a \in S$); and
- (iii) every $e \in E(C)$ concurrent with some member of S (existing anywhere in the configuration) is already in E .

`SettledAt`(C, v, S): v is allocated with event set $E(v)$ and S is settled against $E(v)$. ***SettledAt***

Clause (iii) looks like a global quantification the runtime could never check, and might be carried as a hypothesis the runtime would somehow certify. It is instead a *discharged theorem*: a purely local, checkable frontier predicate implies it.

Definition 18.3 (evidence commits; the frontier predicates). `EvidenceCommit`(C, v, S, j): some version c reaching v (along parents) is allocated with event set $E(c) \supseteq S$ and certifies replica j by the chain clause: every j -authored event outside $E(c)$ has all of S in its causal past ($\forall e \in E(C), \text{rep}(e) = j, e \notin E(c)$ imply $\forall a \in S, a \xrightarrow{\text{vis}} e$). With `Registered`(C, j) (replica j has a head state):

$$\text{AllHeardSince}(C, v, S) :\iff \forall j. \text{Registered}(C, j) \Rightarrow \text{EvidenceCommit}(C, v, S, j).$$

Separately, `AllHeard`(C, S), the all-heads frontier: every current head’s event set contains S (as a gate alone this is the naive contract and unsound, Proposition 18.1; it enters only as the stability side of the runtime contract). ***AllHeardSince, AllHeard***

Theorem 18.4 (the evidence discharge). *Let C satisfy the reachability bundle ReachInvE : (i) the store invariant of Definition 4.1, (ii) every event author is registered, (iii) the root replica 0 is registered; the bundle is supplied by plain or honest reachability ($\text{reachInvE_reachable}$, $\text{reachInvE_honestReach}$). If v is allocated, S is causally downward closed (the hypothesis the runtime must hand: its cut is a downward-closed set), and $\text{AllHeardSince}(C, v, S)$, then $\text{SettledAt}(C, v, S)$. $\text{settledAt_of_allHeard}$*

The argument is the evidence one made rigorous: an event e concurrent with some $s \in S$ was minted by a replica j before j saw s ; the absorbed evidence commit c_j has $s \in E(c_j)$, so e precedes c_j at j and $e \in E(c_j) \subseteq E(v)$. The impossible case (were e still outside $E(v)$) would place the cut event s in e 's minter's past, contradicting concurrency. This is causal stability in the sense of Baquero, Almeida, and Shoker, transplanted from op-based delivery to the commit DAG: there the argument rides on causal delivery order, here on downward closure of event sets. Three facts make it a genuine discharge rather than a relabeled assumption. The frontier the runtime maintains — the per-replica latest-absorbed commit, meeting to the largest certifiable cut — is tracked *axiom-free*: no choice, no classical step enters the version-level payoff. The gate is *SettledAt* and *all-heard*: settledness alone is not stable under future mints by replicas that have not yet heard of S , so the all-heads frontier (the same one §17 tracks) keeps it stable, and *createReplica* is the *one* breaker — a replica joining after a compaction has heard nothing since S , falsifying *AllHeardSince* until it catches up, which is exactly the open-membership gate (a side condition of the theorem, not an unproved promise). And evidence is *commit-shaped*, not event-shaped: a pull that brings no new events can still bring the evidence commit that opens the gate, so a runtime that skips event-subsumed pulls silently starves the callback.

Two species of compaction. *Drop* (OR-set, MVR, remove-record GC): **compact** removes records; under a live-set merge rule a removed record propagates exactly like a deletion, so compaction spreads through merges with no protocol. *Remap* (the embedded-chain RGA's coordinate re-coding, §): **compact** rewrites coordinates by an order isomorphism below the cut, and mixed states need translation on ingest. One VC skeleton covers both, with the simulation relation R_S instantiated to “differ only by S -redundant records” for the drop species and to the graph of the coordinate isomorphism for the remap species.

The six VCs.

Definition 18.5 (the stability VC bundle). *Fix a conditioned MRDT. A stability bundle consists of data and six proof obligations. The data: a cut S ; a compaction $\text{compact} : \Sigma \rightarrow \Sigma$; a simulation relation $R_S \subseteq \Sigma \times \Sigma$ (full state on the left, compacted on the right; reflexive, so the pre-compaction run is related); a read interface $\text{obs} : \Sigma \rightarrow \text{Obs}$; an auxiliary pair invariant Aux on configuration pairs; a compaction gate, which must imply *SettledAt* at the gated version (field `gate_settled`); and a replica-creation policy `canCreate`. The obligations:*

S1 Entry (*vc_entry*). *Under the lifted relation and Aux , at a gated version v (a head, allocated on both sides with the shared event set), $\text{mergeL}(\sigma_v, \sigma_v, \sigma_v) R_S \text{compact}(\hat{\sigma}_v)$: the full side's stutter payload against the compacted callback.*

S2 Observation (*vc_obs*). $\sigma R_S \tau$ implies $\text{obs}(\sigma) = \text{obs}(\tau)$.

S3 *Step preservation (vc_step).* Under the lifted relation and Aux, at a related head (v allocated on both sides with the shared event set, states $\sigma R_S \hat{\sigma}$) and a store-wide fresh timestamp t (no observed event, and no event of any registered version, carries t): $\text{do}(\sigma, e) R_S \text{do}(\hat{\sigma}, e)$ for $e = (t, r, o)$: any op minted with S in its causal past (all future ops, by stability). A second class could arise here, in-flight ops referencing dropped records; in the state-based ternary model those effects arrive only through merge branch states, so the class is absorbed into S_4 and the step VC covers new mints alone.

S4 *Merge congruence, argumentwise and reachability-indexed (vc_merge).* Under the lifted relation and Aux, at genuine merge premises (heads v_1, v_2 , an lsLCA witness v_T , all three registered on both sides with shared event sets): $\text{mergeL}(\sigma_T, \sigma_1, \sigma_2) R_S \text{mergeL}(\hat{\sigma}_T, \hat{\sigma}_1, \hat{\sigma}_2)$, so that mixed executions (a compacted head merging against a full LCA payload, or against a replica that never compacted) stay related. Argumentwise is forced, because mixed is the normal case. And the congruence is not free: unconditioned, it is false for the OR-set instance (a related LCA pair, an A past a remove that killed the kept twin, and a B still carrying the dropped instance live make the two merges read differently). The offending triple is unreachable (any state descending from the compacted LCA sheds the drop at its first absorption merge), which is exactly why the VC fires only under the threaded Aux, not as a free equation.

S5 *Contract preservation (vc_inv).* R_S preserves Inv : $\sigma R_S \tau$ and $\text{Inv}(\sigma)$ imply $\text{Inv}(\tau)$. This is the clause that composes compaction with the conditioned framework, and the one with no analogue in unverified systems. (A wider reading, preservation of the applicable set and of the honesty contract, is not a bundle field; it is recovered observationally by the metatheorem's third conclusion, Theorem 18.7(iii).)

S6 *Epoch coherence, observational (the standalone EpochCoherentObs, across two bundles V, V' with the same read interface):* for every state σ , $\text{obs}(\text{compact}'(\text{compact}(\sigma))) = \text{obs}(\text{compact}'(\sigma))$. That is: compacting at S and then at $S' \supseteq S$ lands in the same observational class as compacting at S' directly, so staggered and repeated compactions across replicas stay related. Observational is the correct strength: as a same- R -class statement it is false (a tag dropped at the first epoch whose second-epoch keeper has since died is kept by the direct coarser compaction; reads still agree).

The bundle closes with the Aux-threading obligations (**aux_init**, **aux_create**, **aux_apply**, **aux_merge**, **aux_compact**): Aux holds initially and is preserved by each matched step (creation gated by **canCreate**) and by the gated compaction. **StabilityVC**

The metatheorem, and the stuttering-self-merge trick.

Definition 18.6 (the lifted relation, and the paired run). $\text{ConfigRelS } R \ C \ \hat{C}$ (**ConfigRelS**): the two configurations share L , $\xrightarrow{\text{vis}}$, head, and parents verbatim; on the full side heads are allocated and headless replicas are stateless (two reachability facts carried in the relation, so the matching lemmas close without a reachability detour); event sets agree slot for slot (ver-projections to event sets equal); and payloads are pointwise related: $\text{ver}_C(v) = (s, E)$ and $\text{ver}_{\hat{C}}(v) = (\hat{s}, \hat{E})$ imply $R \ s \ \hat{s}$. The paired run $\text{StabReach } V$ (**StabReach**, for a bundle V and $\text{Inv}(\sigma_0)$): from the initial pair, either a matched Step3 pair with mutating labels matched verbatim (queries may

answer each side's own value), or a gated compaction (the full side takes the stuttering self-merge $\text{merge}(r, r)$, the compacted side the callback move), each case carrying the re-established $\text{ConfigRelS} \wedge \text{Aux}$ (supplied by the matching lemmas *stability_match* / *stability_compact*).

Theorem 18.7 (the stability metatheorem). *Along every paired run $\text{StabReach } V \ C \ \hat{C}$:*

- (i) $\text{ConfigRelS } V.R \ C \ \hat{C}$ and $V.\text{Aux } C \ \hat{C}$ hold, and the full side C is ordinary **Step3-reachable** (compactions project to stuttering self-merges): *stability_simulation*;
- (ii) reads are equal at every version: $\text{ver}_C(v) = (s, E)$ and $\text{ver}_{\hat{C}}(v) = (\hat{s}, \hat{E})$ imply $\hat{E} = E$ and $\text{obs}(\hat{s}) = \text{obs}(s)$: *stability_reads_equal*;
- (iii) *RA-linearizability is inherited observationally*: if C is *RA-linearizable per version* (Definition 3.4), then every version $(\hat{s}, \hat{E})$ of $\hat{C}$ admits an enumeration π of $\hat{E}$ respecting the linearization order of $\hat{C}$'s own core with $\text{obs}(\text{fold}(\sigma_0, \pi)) = \text{obs}(\hat{s})$: *stability_ra_inherited*.

Axioms: propext, Quot.sound.

The proof device worth recording: compaction projects, on the full side, to a *stuttering self-merge*. The LCA predicate is reflexive and $E \cup E = E$, so merging a head with itself is a legal step that allocates a new version with the same event set; the full run performs that stutter exactly where the compacted run compacts, and the twin DAGs stay *literally identical* in shape, so no version-map bisimulation is ever needed. The device is mechanized as the reflexive LCA and the self-merge configuration (*isLCA_self*, *selfMergeCfg*, *stability_compact*). That one trick is why the metatheorem's axiom bill is $\{\text{propext}, \text{Quot.sound}\}$: no choice, no classical reasoning, just the step relation walked in lockstep.

One consequence deserves its own sentence, because it constrains future representation work: *compaction makes the payload no longer a function of the event set*. Resolving an LCA by event set alone is sound before compaction and unsound after (a same-event-set version from an unrelated lineage used as the LCA slot resurrects a dropped instance); the framework's structural LCA already resolves within shared ancestry, so the model is right as it stands, but any representation swap that identifies versions by clocks or event sets must retain enough lineage, or restrict lookup to R_S -compatible payloads.

First instance: the OR-set, and the refuted static drop. The redundancy predicate: among the *live* instances of an element, the adds in S that are not the kept witness. Under *SettledAt* every remove that could discriminate between two S -instances is already applied (that is Figure 7 inverted), so the surviving S -instances live and die together from here on, and dropping all but a live kept witness preserves every read. Two mechanization findings. The drop set must be *state-dependent* even under the settled gate:

Proposition 18.8 (the static drop is unsound). *At the settled state s_{afterRem} (the kept twin (t_2, a) already dead before compaction fires, only (t_1, a) live), the static, twin-blind drop computed once from the cut erases the element's only surviving instance:*

$$\text{compact}_{\text{static}}(s_{\text{afterRem}})(t_1, a) = \text{false} \quad \wedge \quad a \notin \text{read}(\text{compact}_{\text{static}}(s_{\text{afterRem}})),$$

*while the twin-liveness-guarded callback refuses and is the identity there (*settled_refusal_id*). Newest-first is not needed for soundness: any live kept witness works. *static_drop_unsound**

And the interesting S4 case is a remove minted elsewhere against a full state whose kill set names a dropped instance: on the compacted side the kill is a partial no-op, and the states stay related because the dropped instance's fate is determined by the kept one. The instance discharges all six clauses:

Theorem 18.9 (the OR-set stability instance). *For any cut specification ($\text{CutSpec } S \ K \ nt$: K marks the droppable S -adds, nt names each marked tag's kept newer twin, with the coherence clauses tying both to S : marked tags and their twins are S -adds of the same element, twins are strictly newer and themselves unmarked), the bundle with $R_S :=$ “differ only by S -redundant records” (orRel), $\text{obs} :=$ the membership read, $\text{compact} :=$ the twin-liveness-guarded drop (orCompactC), and $\text{gate} := \text{SettledAt} \wedge \text{AllHeard}$ discharges all six VCs of Definition 18.5. Consequently every paired run reads identically at every version ($\text{ORSet_stability_reads}$) and inherits RA-linearizability observationally ($\text{ORSet_stability_ra}$), by Theorem 18.7. orStabilityVC*

The remap species' standing cut hypothesis is discharged by the same gate ($\text{eRecode_settled_bridge}$, the bridge § consumes).

Part II

Audit and mechanization scope

19 Mechanization scope

The Lean development for this paper is rooted at `Sal/ConditionedMRDTs/Framework/` and `Sal/ConditionedMRDTs/Metatheory/`. It includes the executable countermodels in `Refutations/`, the flat and conditioned instance discharges in `MRDT_Instances/`, and the product, quotient, virtual-LCA, GC, and stability modules described in the body. The cited proof chains use no `sorryAx`; the repository’s verification companion records theorem names, files, and axiom audits. Open work is tracked durably in the repository roadmap rather than mixed into the paper’s claims.